Laporan Praktikum Sistem Basis Data - Section 1-4

Dosen Pengampu: Rizal Broer Bahaweres, M.Kom.



Disusun oleh:

Yafi Abhiyasa Ichsan 1251420025

PRODI TEKNIK INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UIN SYARIF HIDAYATULLAH JAKARTA

2026

1. SECTION-0 Kursus & Lingkungan Belajar (Course Overview & Environment Setup)

A. Penjelasan

Section 0 merupakan pengenalan awal terhadap ruang lingkup modul pembelajaran Database Foundations yang disediakan oleh Oracle Academy. Pada bagian ini, mahasiswa diperkenalkan dengan lingkungan kerja (environment) yang akan digunakan selama proses pembelajaran, baik yang bersifat konseptual (pemodelan data) maupun teknis (pemrograman SQL).

Poin-Poin Utama:

1. **Tujuan Kurikulum:** Memahami dasar-dasar perancangan basis data relasional, mulai dari konseptual (ERD) hingga implementasi fisik menggunakan *Structured Query Language* (SQL).
2. **Lingkungan Kerja Utama:**
    - **Oracle APEX (Application Express):** Platform berbasis *web browser* yang digunakan untuk menulis, mengeksekusi, dan menguji perintah-perintah SQL (DDL, DML, TQL) secara *hands-on*.
    - **Oracle SQL Developer Data Modeler:** Perkakas (*tools*) visual desktop yang digunakan untuk merancang dan memetakan *Entity Relationship Diagram* (ERD).
3. **Pemisahan Kategori Tugas:**
    - *Conceptual & Analytical Practice:* Penugasan analisis studi kasus dan teks di lembar kerja / Word (Section 1 - Section 4).
    - *Hands-on / Technical Practice:* Penugasan pengetikan baris kode SQL di Oracle APEX SQL Workshop (Section 5 - Section 6 dan seterusnya).

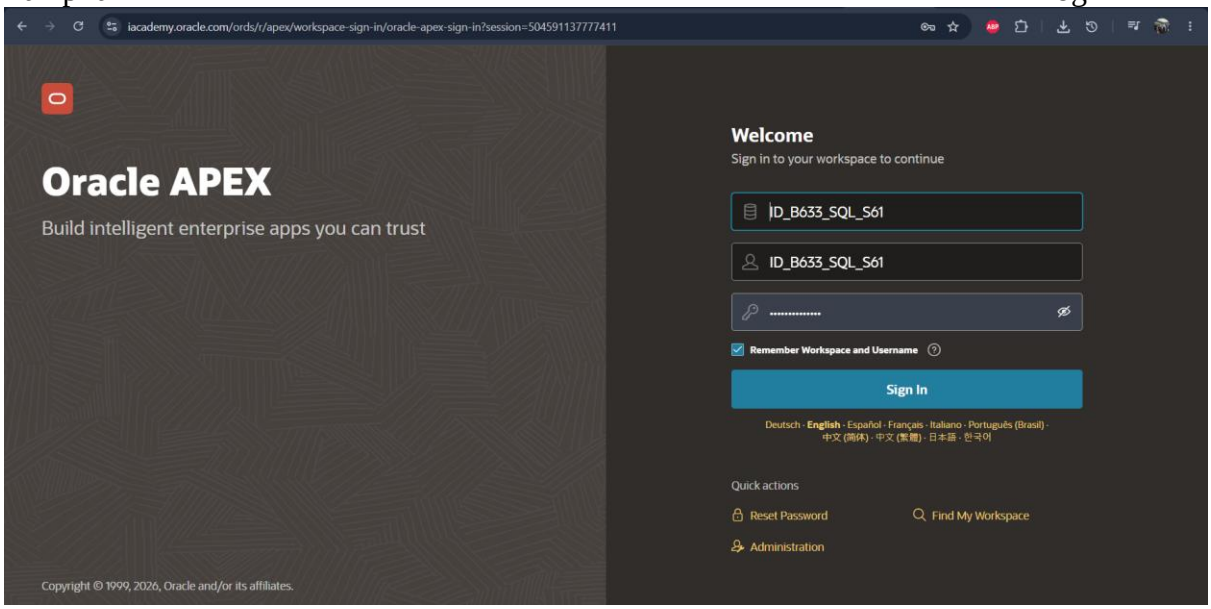
B. Bentuk Coding, Pratikum, Gambar, dan Penggunaan Oracle Apex

Pada Section 0, fokus utama praktikum adalah melakukan verifikasi dan uji akses akun pada lingkungan Oracle APEX Workspace.

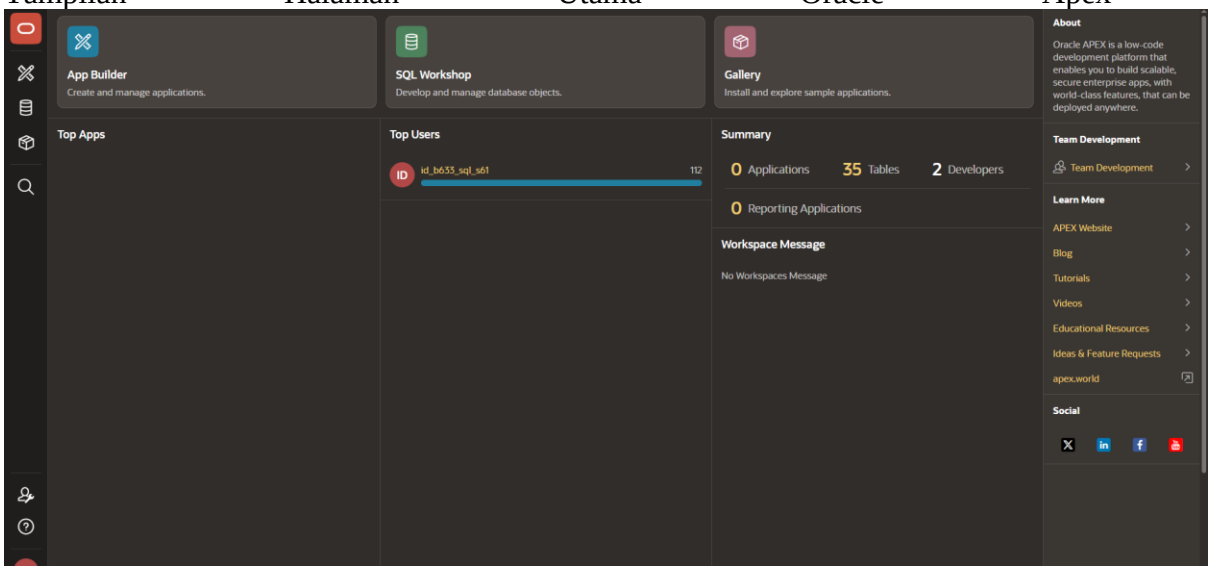
Langkah Kerja Praktikum:

1. Membuka peramban (*web browser*) dan mengakses URL Oracle APEX.
2. Mengisikan credentials: **Workspace Name**, **Username**, dan **Password**.
3. Menjelajahi antarmuka utama (*Dashboard*), khususnya menu **SQL Workshop** > **SQL Commands** dan **Object Browser**.
4. Download file sql yang sudah disediakan

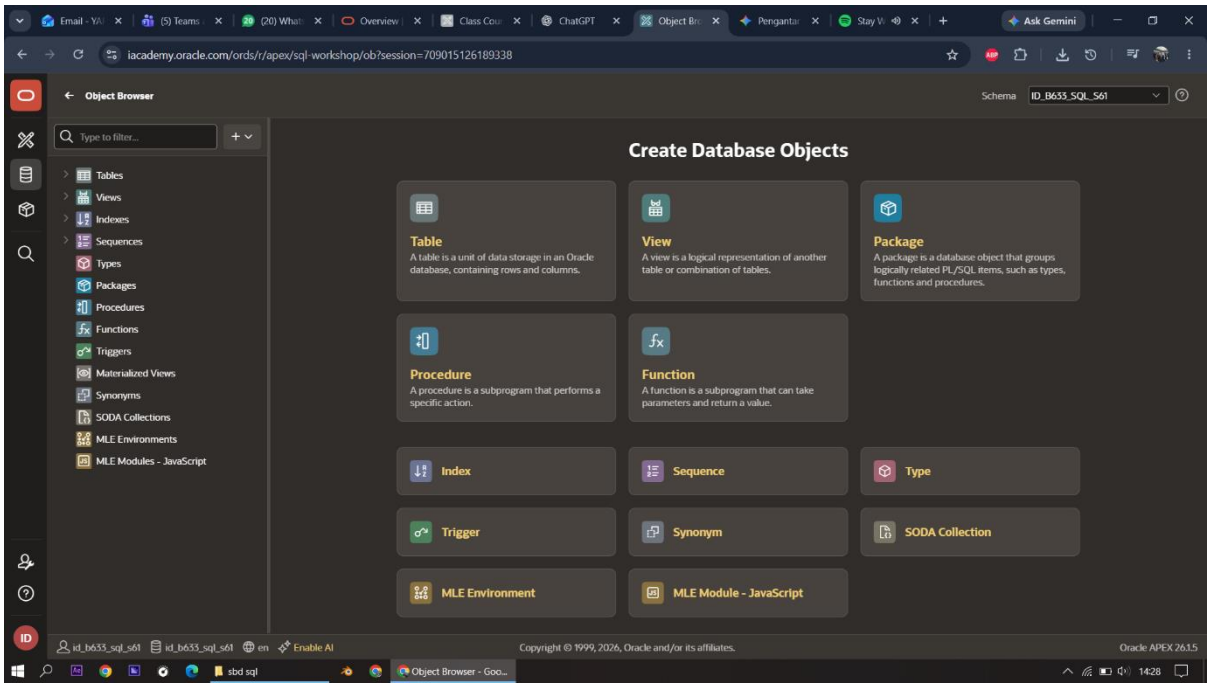
Tampilan Login



Tampilan Halaman Utama Oracle Apex



Tampilan Dashboard SQL Workshop Sql Object dan tampilan file scheme_sql



C. Catatan Penting & Identifikasi Masalah Umum

- Kendala Akses Login: Pastikan nama Workspace sesuai dengan format yang telah didaftarkan oleh pak broer

- Skema Basis Data (Schema): Pastikan skema aktif di pojok kanan atas layar SQL Commands sudah terhubung ke skema pengguna yang tepat agar tidak terjadi kesalahan akses objek.

2. SECTION-1 Pengenalan Basis Data & Analisis Kebutuhan Bisnis (Introduction & Business Requirements)

A. Penjelasan

1.1 Introduction to the Course

Introduction to the Course merupakan materi pengenalan mengenai pembelajaran Database Foundations. Materi ini bertujuan memberikan pemahaman dasar mengenai database, perancangan database, penggunaan Entity Relationship Diagram (ERD), serta penerapan SQL menggunakan Oracle Application Express (APEX).

Tujuan pembelajaran pada bagian ini meliputi:

1. Memahami tujuan dan materi pembelajaran Database Foundations.
2. Mengetahui konsep dasar relational database.
3. Memahami proses perancangan database menggunakan data modeling.
4. Mengenal penggunaan ERD untuk menggambarkan struktur database.
5. Memahami pembuatan physical model dari logical model.
6. Mampu menulis, menjalankan, dan menyimpan perintah SQL di Oracle APEX.

Tools yang digunakan dalam pembelajaran ini meliputi Oracle SQL Developer Data Modeler dan Oracle Application Express (APEX).

1.2 Introduction to Databases

Database merupakan kumpulan data yang tersusun dan tersimpan secara terstruktur di dalam sistem komputer. Database digunakan untuk menyimpan, mengambil, menambahkan, mengubah, serta menghapus data sesuai kebutuhan.

Beberapa konsep dasar dalam database:

1. Data dan Information

- Data adalah fakta yang dikumpulkan mengenai suatu objek atau topik.
- Information merupakan hasil pengolahan, penggabungan, perbandingan, atau perhitungan dari data sehingga menghasilkan informasi yang berguna.

2. Database

Database merupakan tempat penyimpanan data terpusat dan terstruktur yang memungkinkan pengguna mengelola informasi dengan lebih mudah.

3. Relational Database

Relational Database merupakan database yang menyimpan informasi dalam bentuk tabel yang terdiri dari baris dan kolom.

- Table: Kumpulan data atau record.
- Row: Baris yang berisi sebuah record.
- Column: Kolom yang menggambarkan field atau attribute.

4. Database Management System (DBMS)

DBMS merupakan perangkat lunak yang digunakan untuk mengatur penyimpanan, pengorganisasian, dan pengambilan data.

Elemen DBMS meliputi:

- Memory and Storage Management.
- Data Dictionary.

- Query Language.

5. Transformations in Computing

Perkembangan sistem komputasi dalam materi meliputi:

- 1970s: Mainframe Computing.
- 1980s: Desktop Computing.
- 1990s: Client/Server Computing.
- 2000s: Grid Computing.
- 2010s: Cloud Computing.

1.3 Types of Database Models

Database Model merupakan cara atau metode untuk menggambarkan bagaimana data disusun, disimpan, dan saling berhubungan dalam suatu sistem database.

Jenis-jenis database model yang dipelajari:

1. **Flat File Model:** Menyimpan data dalam satu struktur file atau tabel sederhana tanpa hubungan antarentitas seperti pada relational database.
2. **Hierarchical Model:** Menyusun data dalam bentuk hierarki atau struktur pohon yang memiliki hubungan parent dan child.
3. **Network Model:** Menghubungkan data dalam bentuk jaringan sehingga satu data dapat memiliki hubungan dengan beberapa data lainnya.
4. **Object-Oriented Model:** Menyimpan data dalam bentuk objek yang memiliki atribut dan metode.
5. **Relational Model:** Menyimpan data dalam tabel yang dapat dihubungkan melalui atribut atau key yang sama.

1.4 Business Requirements

Business Requirements merupakan kebutuhan atau ketentuan yang harus dipahami sebelum suatu database dirancang dan dikembangkan.

Business Requirements digunakan untuk memahami proses bisnis, kebutuhan data, serta aturan yang harus diterapkan pada sebuah sistem.

Terdapat tiga kategori utama:

1. Business Rule

Aturan yang menjelaskan bagaimana suatu proses bisnis harus berjalan dan ketentuan yang harus dipenuhi.

2. Assumption

Asumsi atau pernyataan yang dianggap benar sementara dalam proses perancangan, tetapi masih dapat membutuhkan konfirmasi.

3. Problem

Permasalahan atau kondisi yang membutuhkan perhatian dan solusi agar sistem dapat berjalan dengan baik.

B. Bentuk Coding, Pratikum, Gambar, dan Penggunaan Oracle Apex

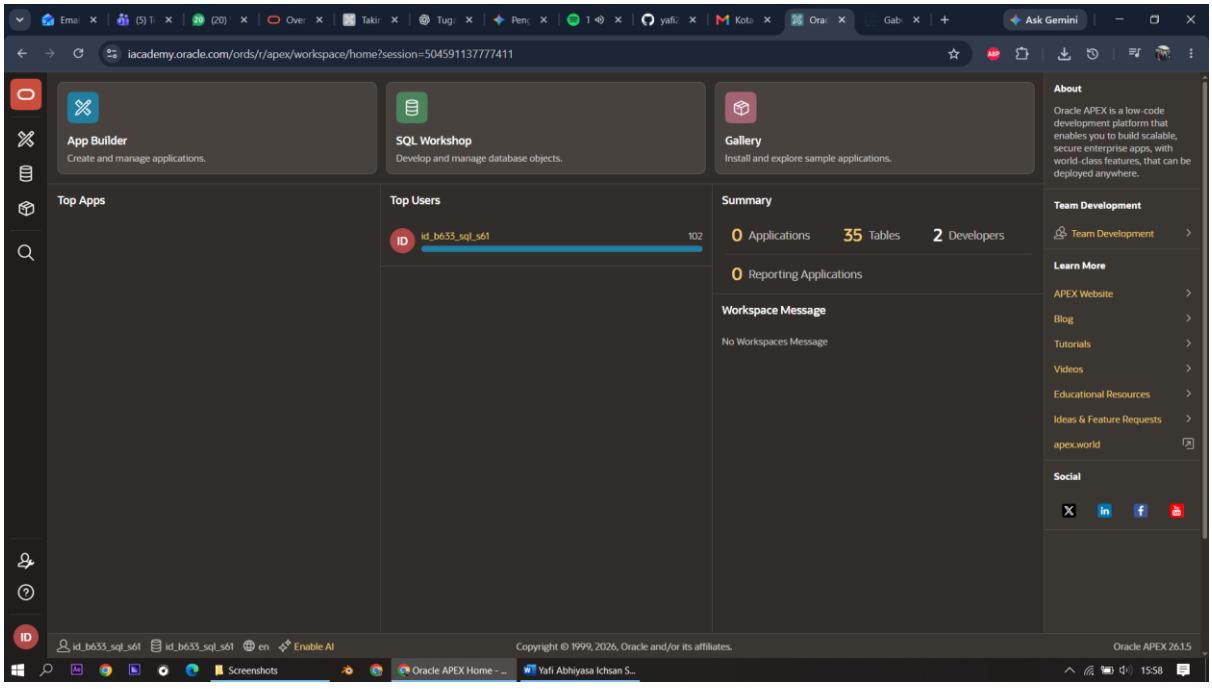
1.1 Introduction to the Course

Pada pembelajaran ini tidak terdapat exercise khusus yang meminta pembuatan database.

Dokumentasi yang diperlukan:

- Screenshot materi pengenalan Database Foundations.
- Screenshot halaman workspace Oracle APEX.
- Penjelasan singkat tools yang digunakan.

Gambar 1.1: Tampilan awal Oracle APEX.



1.2 Introduction to Databases

Exercise 1: Identify Database Design Considerations for Given Case Scenarios

Pada praktikum ini terdapat dua studi kasus yang membutuhkan identifikasi data penting untuk membangun database.

A. Student Registration System

Studi kasus ABC School District membutuhkan sistem pendaftaran siswa secara online. Sistem harus memungkinkan siswa baru melakukan pendaftaran serta siswa yang sudah terdaftar memperbarui dan melihat informasi mereka.

Data yang perlu disimpan:

Kategori	Data Yang Dibutuhkan
Student Identity	Student ID, Full Name, Date of Birth
Contact Information	Address, Email, Phone Number
Academic Information	Previous School, Grade, Academic Year
Registration	Registration ID, Registration Date, Registration Status
Parent/Guardian	Guardian Name, Relationship, Contact Number

B. Library Management System

Studi kasus XYZ Community membutuhkan sistem pengelolaan perpustakaan yang dapat menangani informasi buku, pelanggan, dan aktivitas sehari-hari perpustakaan.

Data yang perlu disimpan:

Kategori	Data Yang Dibutuhkan
Book Data	Book ID, ISBN, Book Title, Author, Publisher, Publication Year, Category
Customer Data	Member ID, Customer Name, Address, Email, Phone Number
Borrowing Data	Borrowing ID, Book ID, Member ID, Borrow Date, Due Date
Return Data	Return Date, Borrowing Status
Library Activities	Reservation Information, Fine Information, Transaction History

C. Implementasi Menggunakan Oracle APEX

Untuk menerapkan hasil identifikasi data, dilakukan pembuatan tabel database menggunakan SQL pada Oracle APEX.

Rancangan sederhana yang digunakan:

Student Registration System:

- STUDENT
- Kolom identitas siswa dan informasi pendaftaran.

Library Management System:

- BOOK
- MEMBER
- BORROWING

Tabel BORROWING dapat digunakan untuk menyimpan transaksi peminjaman dengan menghubungkan data MEMBER dan BOOK.

1.3 Types of Database Models

Exercise 1: Identify the Database Models

Pada praktikum ini dilakukan identifikasi lima gambar untuk mengetahui jenis database model yang digunakan.

Hasil identifikasi:

Gambar	Data Model	Penjelasan
A	Hierarchical Model	Data Employee disusun dalam bentuk hierarki yang memiliki hubungan parent dan child.
B	Network Model	Customer memiliki hubungan dengan beberapa Bank Account melalui struktur jaringan.
C	Object-Oriented Model	Data Employee digambarkan sebagai class yang memiliki attributes dan methods serta objek yang merupakan instance dari class tersebut.
D	Flat File Model	Data Customer disimpan dalam satu tabel sederhana yang berisi Account ID, Customer Name, dan Branch.
E	Flat File Model	Data buku disimpan dalam satu tabel sederhana yang berisi Book ID, Author Name, dan Book Title.

Implementasi Oracle APEX

Exercise ini berfokus pada identifikasi jenis database model berdasarkan gambar. Tidak terdapat instruksi untuk membuat lima model database berbeda menggunakan SQL.

Sebagai penerapan sederhana, tabel CUSTOMER atau BOOK dapat dibuat menggunakan SQL untuk memperlihatkan penyimpanan data dalam bentuk tabel.

[Masukkan coding SQL apabila dilakukan]

[Masukkan screenshot hasil tabel apabila dilakukan]

1.4 Business Requirements

Pada pembelajaran ini dilakukan analisis terhadap dua studi kasus untuk mengidentifikasi business rules dan constraints.

A. LibBook Digital Library

LibBook merupakan perpustakaan digital yang menyediakan penyewaan CD dan akses internet untuk membaca artikel serta majalah.

Berdasarkan kasus tersebut, business requirements yang dapat diidentifikasi:

Business Rules:

1. LibBook menerapkan sistem membership untuk para anggotanya.
2. Terdapat tiga jenis membership awal, yaitu Corporate, Student, dan Individual.
3. Membership Student diberikan secara gratis.
4. Jenis membership Corporate dan Faculty disebutkan memiliki biaya serta memberikan hak istimewa.
5. Jenis membership dapat diubah apabila terdapat alasan atau justifikasi yang memadai.
6. Sistem harus mendukung penambahan jenis membership baru apabila diperlukan.

Constraints:

1. Perubahan jenis membership harus disertai justifikasi yang memadai.
2. Membership Student tidak dikenakan biaya.
3. Membership Corporate dan Faculty dikenakan biaya sesuai ketentuan.
4. Hak istimewa diberikan berdasarkan jenis membership.

Problem yang Perlu Diperhatikan:

- Jumlah anggota tidak stabil meskipun pertumbuhan anggota cukup cepat.
- Ketentuan biaya untuk membership Individual belum dijelaskan.

- Terdapat perbedaan penyebutan jenis membership, yaitu Individual pada daftar awal tetapi Faculty pada penjelasan biaya.

  Catatan: Perbedaan Individual dan Faculty tersebut perlu dikonfirmasi agar rancangan database tidak menggunakan aturan yang keliru.

B. Star Care Hospital

Star Care merupakan rumah sakit multi-spesialisasi yang membutuhkan pengelolaan informasi dokter dan pasien.

Business Rules:

1. Setiap dokter yang terdaftar memiliki ID unik.
2. ID dokter harus diawali dengan huruf DC.
3. Setiap dokter harus mempunyai pengalaman kerja minimal tujuh tahun.
4. Setiap pasien wajib melakukan registrasi pada kunjungan pertama.
5. Setiap pasien akan mendapatkan nomor pasien unik yang diawali dengan huruf PT.

Constraints:

1. Doctor ID wajib unik dan menggunakan awalan DC.
2. Pengalaman kerja dokter minimal tujuh tahun.
3. Patient Number wajib unik dan menggunakan awalan PT.
4. Pasien harus terdaftar sebelum menggunakan layanan rumah sakit.

C. Oracle Baseball League (OBL) Project

Oracle Baseball League merupakan sebuah toko perlengkapan baseball yang membutuhkan rancangan database untuk mengelola pelanggan, tim, produk, pesanan, dan aktivitas penjualan.

2. Assumptions

Berikut merupakan asumsi yang dapat digunakan sebagai dasar awal perancangan database dan masih perlu dikonfirmasi:

1. Setiap pelanggan memiliki Customer ID yang unik.
2. Setiap tim memiliki Team ID dan informasi jumlah pemain.
3. Setiap produk memiliki Product ID, nama produk, dan harga.
4. Setiap pesanan memiliki Order ID yang unik.
5. Setiap pesanan terhubung dengan data pelanggan.
6. Setiap detail pesanan menyimpan produk dan jumlah barang yang dipesan.
7. Informasi jumlah pemain tim diperlukan untuk menentukan diskon.

3. Problems

1. Besaran dan perhitungan diskon tim belum dijelaskan secara rinci.
2. Batas jumlah pemain untuk setiap tingkatan diskon belum ditentukan.

3. Informasi lengkap pelanggan dan tim belum dijelaskan.
4. Mekanisme pencatatan pesanan dan detail barang perlu ditentukan.
5. Pembagian tanggung jawab sales representative dalam menangani pelanggan individu belum dijelaskan secara terperinci.

4. Kesimpulan Analisis OBL

Berdasarkan studi kasus Oracle Baseball League, database yang dirancang harus mampu menyimpan informasi pelanggan individu dan tim, data inventaris produk, transaksi pesanan, detail pesanan, serta informasi sales representative.

Business rules menjadi dasar untuk menentukan bagaimana data akan saling berhubungan. Assumptions digunakan untuk membantu proses perancangan awal, sedangkan Problems menunjukkan bagian yang masih membutuhkan penjelasan lebih lanjut.

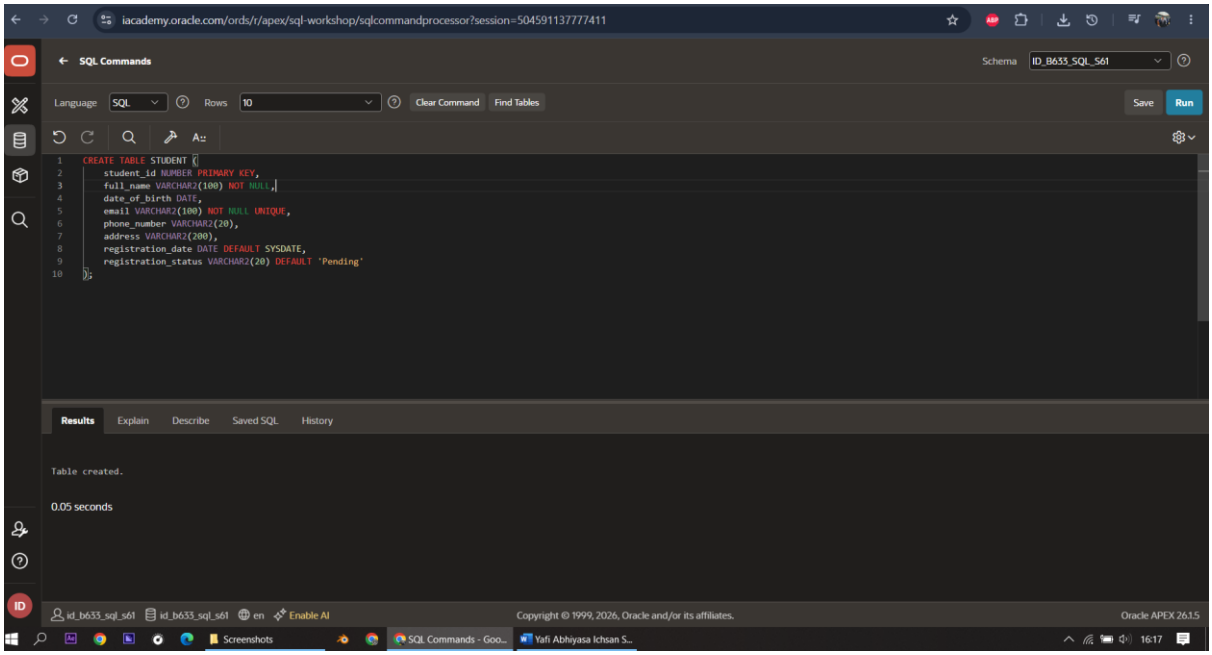
Rancangan tersebut dapat dikembangkan menjadi beberapa tabel, misalnya:

- CUSTOMER
- TEAM
- PRODUCT
- SALES_ORDER
- ORDER_ITEM
- SALES_REPRESENTATIVE

Pembuatan tabel secara lengkap dapat dilanjutkan setelah kebutuhan, aturan diskon, dan hubungan antarentitas sudah ditentukan.

Codingan Oracle Apex

Membuat table Student



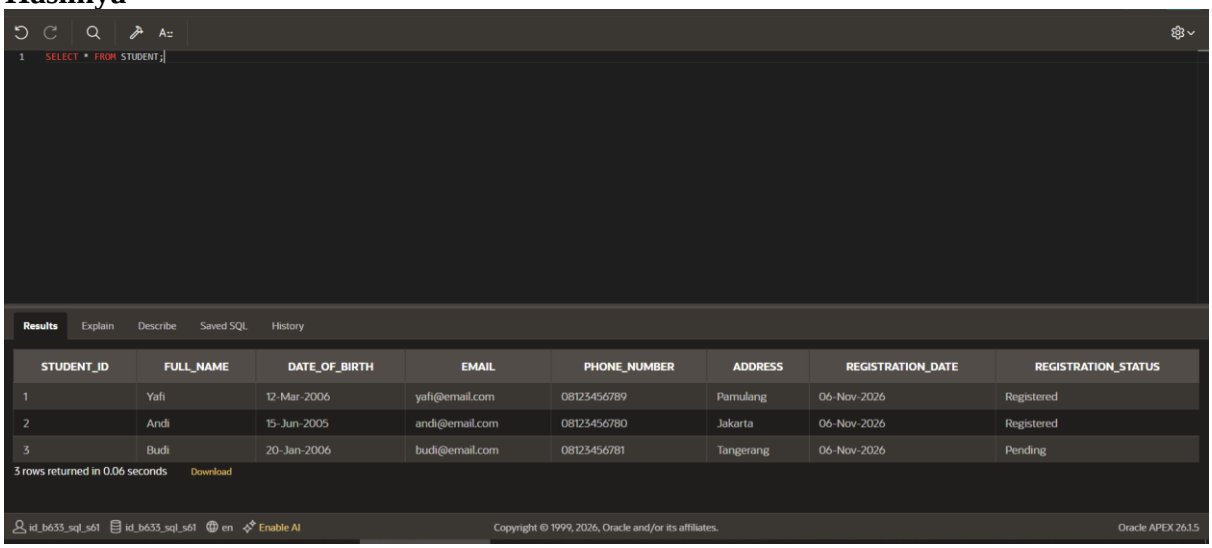
Data Siswa

```
INSERT INTO STUDENT VALUES (
    1, 'Yafi', DATE '2006-03-12',
    'yafi@email.com', '08123456789',
    'Pamulang', SYSDATE, 'Registered'
);


INSERT INTO STUDENT VALUES (
    2, 'Andi', DATE '2005-06-15',
    'andi@email.com', '08123456780',
    'Jakarta', SYSDATE, 'Registered'
);
```

```
INSERT INTO STUDENT VALUES (
    3, 'Budi', DATE '2006-01-20',
    'budi@email.com', '08123456781',
    'Tangerang', SYSDATE, 'Pending'
);
```

Hasilnya



Membuat tabel BOOK

```
CREATE TABLE BOOK (
    book_id NUMBER PRIMARY KEY,
    book_title VARCHAR2(150) NOT NULL,
    author_name VARCHAR2(100),
    category VARCHAR2(50),
```

```
    available_copies NUMBER

);
```

Membuat table library_member

```
CREATE TABLE LIBRARY_MEMBER (

    member_id NUMBER PRIMARY KEY,

    member_name VARCHAR2(100) NOT NULL,

    email VARCHAR2(100) UNIQUE,

    phone_number VARCHAR2(20)

);
```

Membuat table library borrowing

```
CREATE TABLE BORROWING (

    borrowing_id NUMBER PRIMARY KEY,

    book_id NUMBER REFERENCES BOOK(book_id),

    member_id NUMBER REFERENCES LIBRARY_MEMBER(member_id),

    borrow_date DATE DEFAULT SYSDATE,

    due_date DATE NOT NULL,

    return_date DATE

);
```

Masukan data buku

```
INSERT INTO BOOK VALUES

```

```
(1, 'Database Fundamentals', 'John Smith',
 'Technology', 5);



INSERT INTO BOOK VALUES
(2, 'Learning SQL', 'Alan Beaulieu',
 'Programming', 3);
```

Masukan data anggota

```
INSERT INTO LIBRARY_MEMBER VALUES
(1, 'Yafi', 'yafi@email.com', '08123456789');



INSERT INTO LIBRARY_MEMBER VALUES
(2, 'Andi', 'andi@email.com', '08123456780');
```

Masukan transaksi peminjaman

```
INSERT INTO BORROWING (
    borrowing_id, book_id, member_id, due_date
) VALUES (
    1, 1, 1, DATE '2026-11-20'
);
```

Menampilkan hasil

```
SELECT * FROM BOOK;
```

Results | Explain | Describe | Saved SQL | History

BOOK_ID	BOOK_TITLE	AUTHOR_NAME	CATEGORY	AVAILABLE_COPIES
1	Database Fundamentals	John Smith	Technology	5
2	Learning SQL	Alan Beaulieu	Programming	3

2 rows returned in 0.02 seconds Download

```
SELECT * FROM LIBRARY_MEMBER;
```

Results | Explain | Describe | Saved SQL | History

MEMBER_ID	MEMBER_NAME	EMAIL	PHONE_NUMBER
1	Yafi	yafi@email.com	08123456789
2	Andi	andi@email.com	08123456780

2 rows returned in 0.01 seconds Download

```
SELECT * FROM BORROWING;
```

Results | Explain | Describe | Saved SQL | History

BORROWING_ID	BOOK_ID	MEMBER_ID	BORROW_DATE	DUE_DATE	RETURN_DATE
1	1	1	06-Nov-2026	20-Nov-2026	-

Melihat transaksi dan bukunya

```
SELECT
    B.borrowing_id,
    M.member_name,
    K.book_title,
    B.borrow_date,
    B.due_date
FROM BORROWING B
JOIN LIBRARY_MEMBER M
ON B.member_id = M.member_id
JOIN BOOK K
ON B.book_id = K.book_id;
```

Results | Explain | Describe | Saved SQL | History

BORROWING_ID	MEMBER_NAME	BOOK_TITLE	BORROW_DATE	DUE_DATE
1	Yafi	Database Fundamentals	06-Nov-2026	20-Nov-2026

1 rows returned in 0.01 seconds Download

C. Analisis Hasil dan Kesimpulan

1. Analisis Hasil

Berdasarkan kegiatan pembelajaran Database Foundations Section 1, dapat dipahami bahwa database merupakan bagian penting dalam pengelolaan informasi pada suatu organisasi.

Pada pembelajaran ini dilakukan identifikasi kebutuhan data untuk Student Registration System dan Library Management System, pengenalan berbagai jenis database model, serta analisis business requirements melalui beberapa studi kasus.

Praktikum juga memberikan pemahaman mengenai bagaimana kebutuhan suatu organisasi dapat diterjemahkan menjadi rancangan penyimpanan data. Penerapan melalui Oracle APEX digunakan untuk mengenal proses pembuatan tabel, memasukkan data, dan menampilkan hasil menggunakan SQL.

2. Kesimpulan

Dari pembelajaran Section 1, dapat disimpulkan bahwa proses pengembangan database dimulai dengan memahami kebutuhan organisasi dan informasi yang harus dikelola.

Identifikasi data, pemilihan model database, serta penentuan business rules merupakan langkah penting sebelum database dikembangkan lebih lanjut.

Dengan mempelajari materi ini, mahasiswa memperoleh dasar untuk melanjutkan pembelajaran mengenai relational database, data modeling, ERD, dan implementasi SQL menggunakan Oracle APEX.

3. SECTION-2 Data Modeling

1. Tujuan Pembelajaran

Section 2 membahas proses pemodelan data untuk memahami bagaimana data disusun dalam sebuah database. Materi ini mencakup Entity, Attribute, Unique Identifier (UID), Relationship, dan Entity Relationship Diagram (ERD).

2. Materi Pembelajaran

2.1 Entities and Attributes

Entity merupakan objek atau sesuatu yang datanya akan disimpan dalam database. Sementara itu, **Attribute** merupakan informasi atau karakteristik yang dimiliki oleh suatu entity.

Dalam project Oracle Baseball League Store Database (OBL), terdapat beberapa entity utama:

- Customer
- Team
- Sales Representative
- Order
- Item
- Inventory List

Setiap entity memiliki atribut yang diperoleh dari kebutuhan sistem dan hasil wawancara.

Atribut juga dapat dibedakan menjadi:

- **Mandatory Attribute:** atribut yang wajib memiliki nilai.
- **Optional Attribute:** atribut yang boleh tidak memiliki nilai.
- **Volatile Attribute:** atribut yang nilainya dapat berubah sewaktu-waktu.

2.2 Unique Identifiers (UID)

Unique Identifier merupakan atribut atau kombinasi atribut yang digunakan untuk membedakan setiap data dalam suatu entity.

Beberapa konsep UID yang dipelajari:

- **Artificial UID:** identitas yang dibuat secara khusus oleh sistem.
- **Composite UID:** identitas yang dibentuk dari gabungan beberapa atribut.
- **Candidate UID:** atribut atau kombinasi atribut yang berpotensi menjadi identitas unik.
- **Secondary UID:** identitas alternatif yang dapat digunakan untuk mengenali data.
- **Primary Key (PK):** UID yang dipilih sebagai identitas utama suatu tabel.

UID penting untuk memastikan setiap data dapat dikenali secara unik.

2.3 Relationships

Relationship merupakan hubungan antara satu entity dengan entity lainnya.

Jenis hubungan yang dipelajari meliputi:

- **One-to-One (1:1):** satu data berhubungan dengan satu data lainnya.
- **One-to-Many (1):** satu data dapat berhubungan dengan banyak data.
- **Many-to-One (M:1):** banyak data berhubungan dengan satu data.
- **Many-to-Many (M):** banyak data dapat berhubungan dengan banyak data.
- **Recursive Relationship:** hubungan suatu entity dengan entity yang sama.

Selain itu, relationship juga memperhatikan nama hubungan, optionality, dan cardinality.

2.4 Entity Relationship Diagram (ERD)

ERD merupakan diagram yang digunakan untuk menggambarkan struktur data dan hubungan antar-entity dalam sebuah sistem database.

Dalam pembuatannya, ERD membantu menunjukkan:

- Entity yang digunakan.
- Attribute dari setiap entity.
- Unique Identifier.
- Hubungan antar-entity.
- Cardinality dan optionality.

ERD dibuat pada tahap pemodelan konseptual dan logis sebelum diterapkan ke dalam database.

3. Project: Oracle Baseball League Store Database

Pada project ini, materi yang telah dipelajari diterapkan untuk merancang database toko perlengkapan baseball.

Tahapan pengerjaan meliputi:

1. Mengidentifikasi entity dan attribute berdasarkan kebutuhan sistem.
2. Menentukan Unique Identifier untuk setiap entity.
3. Menentukan hubungan antar-entity.
4. Menyusun relationship matrix.
5. Membuat ERD sebagai gambaran struktur database.

Hasil akhir: ERD yang menggambarkan entity, attribute, UID, dan hubungan antar-entity dalam Oracle Baseball League Store Database.

4. Praktikum

Practice 2-1

Task 1 : book.com scenario

Book.com merupakan toko buku online yang memungkinkan pelanggan melihat katalog dan membeli buku melalui internet.

Berdasarkan studi kasus tersebut, tabel-tabel yang dibutuhkan adalah:

No.	Nama Tabel	Field
1	Authors	author_id (PK), author_name, address, homepage_url
2	Publishers	publisher_id (PK), publisher_name, address, phone, website_url
3	Books	book_id (PK), title, isbn, publication_year, price, author_id (FK), publisher_id (FK)
4	Warehouses	warehouse_code (PK), address, phone
5	Warehouse_Stock	warehouse_code (PK, FK), book_id (PK, FK), stock_quantity
6	Customers	customer_id (PK), customer_name, address, email, phone
7	Shopping_Carts	cart_id (PK), customer_id (FK), created_at
8	Cart_Items	cart_id (PK, FK), book_id (PK, FK), quantity
9	Customer_Orders	order_id (PK), customer_id (FK), cart_id (FK), order_date, billing_address, shipping_address, shipping_option, payment_reference, payment_status, email_notification_sent_at
10	Order_Items	order_id (PK, FK), book_id (PK, FK), quantity, unit_price

Penjelasan relasi antar tabel:

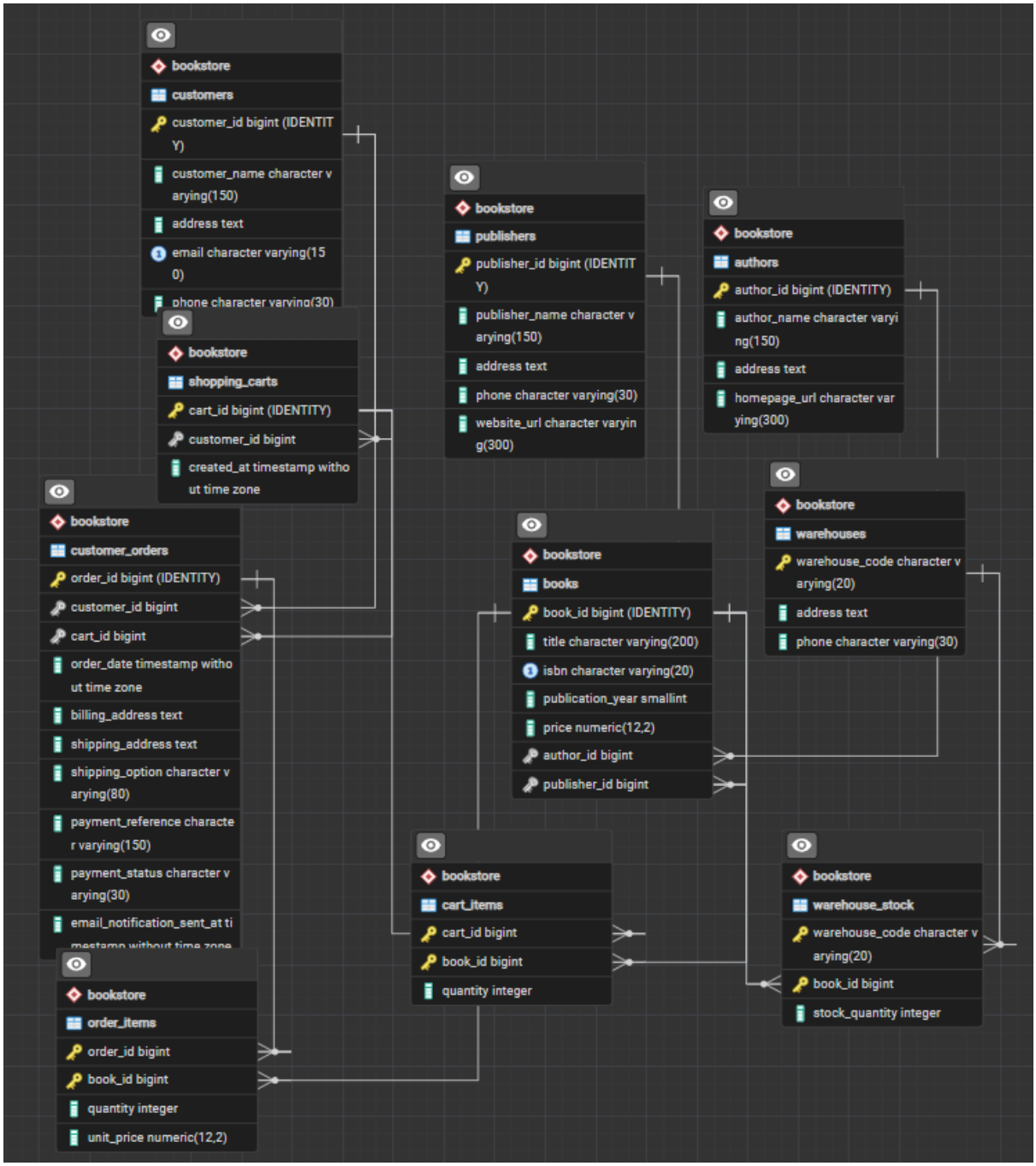
- Authors dengan Books: satu penulis dapat menulis beberapa buku (One-to-Many).
- Publishers dengan Books: satu penerbit dapat menerbitkan beberapa buku (One-to-Many).
- Books dengan Warehouses: satu buku dapat disimpan di beberapa gudang, dan satu gudang menyimpan banyak buku (Many-to-Many). Hubungannya menggunakan Warehouse_Stock.
- Customers dengan Shopping_Carts: satu pelanggan dapat memiliki beberapa keranjang belanja (One-to-Many).
- Shopping_Carts dengan Books: satu keranjang dapat berisi banyak buku dan satu buku dapat berada di banyak keranjang. Hubungannya menggunakan Cart_Items.
- Customers dengan Customer_Orders: satu pelanggan dapat melakukan beberapa pesanan (One-to-Many).
- Customer_Orders dengan Books: satu pesanan dapat berisi beberapa buku. Hubungannya menggunakan Order_Items.

Kesimpulan Exercise 1:

Database Book.com membutuhkan beberapa tabel yang saling berhubungan untuk mengelola informasi buku, penulis, penerbit, gudang, pelanggan, keranjang belanja, dan transaksi pemesanan.

Tabel tambahan seperti Warehouse_Stock, Cart_Items, dan Order_Items digunakan untuk menangani hubungan Many-to-Many serta mencatat jumlah buku yang disimpan atau dibeli.

ERD



Codingan oracle apex

```
CREATE TABLE AUTHORS (
  AUTHOR_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  AUTHOR_NAME VARCHAR2(150) NOT NULL,
  ADDRESS VARCHAR2(500),
  HOMEPAGE_URL VARCHAR2(300)
```

```
);

CREATE TABLE PUBLISHERS (
    PUBLISHER_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    PUBLISHER_NAME VARCHAR2(150) NOT NULL,
    ADDRESS VARCHAR2(500),
    PHONE VARCHAR2(30),
    WEBSITE_URL VARCHAR2(300)
);

CREATE TABLE BOOKS (
    BOOK_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    TITLE VARCHAR2(200) NOT NULL,
    ISBN VARCHAR2(20) UNIQUE NOT NULL,
    PUBLICATION_YEAR NUMBER(4),
    PRICE NUMBER(12,2) NOT NULL,
    AUTHOR_ID NUMBER REFERENCES AUTHORS(AUTHOR_ID),
    PUBLISHER_ID NUMBER REFERENCES PUBLISHERS(PUBLISHER_ID)
);

CREATE TABLE WAREHOUSES (
    WAREHOUSE_CODE VARCHAR2(20) PRIMARY KEY,
    ADDRESS VARCHAR2(500) NOT NULL,
    PHONE VARCHAR2(30)
);

CREATE TABLE WAREHOUSE_STOCK (
    WAREHOUSE_CODE VARCHAR2(20) REFERENCES WAREHOUSES(WAREHOUSE_CODE),
    BOOK_ID NUMBER REFERENCES BOOKS(BOOK_ID),
    STOCK_QUANTITY NUMBER DEFAULT 0,
```

```
    PRIMARY KEY (WAREHOUSE_CODE, BOOK_ID)
);

CREATE TABLE CUSTOMERS (
    CUSTOMER_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    CUSTOMER_NAME VARCHAR2(150) NOT NULL,
    ADDRESS VARCHAR2(500),
    EMAIL VARCHAR2(150) UNIQUE NOT NULL,
    PHONE VARCHAR2(30)
);

CREATE TABLE SHOPPING_CARTS (
    CART_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    CUSTOMER_ID NUMBER REFERENCES CUSTOMERS(CUSTOMER_ID),
    CREATED_AT TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE CART_ITEMS (
    CART_ID NUMBER REFERENCES SHOPPING_CARTS(CART_ID),
    BOOK_ID NUMBER REFERENCES BOOKS(BOOK_ID),
    QUANTITY NUMBER NOT NULL,
    PRIMARY KEY (CART_ID, BOOK_ID)
);

CREATE TABLE CUSTOMER_ORDERS (
    ORDER_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    CUSTOMER_ID NUMBER REFERENCES CUSTOMERS(CUSTOMER_ID),
    CART_ID NUMBER REFERENCES SHOPPING_CARTS(CART_ID),
    ORDER_DATE TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    BILLING_ADDRESS VARCHAR2(500) NOT NULL,
```

```
  SHIPPING_ADDRESS VARCHAR2(500) NOT NULL,
  SHIPPING_OPTION VARCHAR2(80),
  PAYMENT_REFERENCE VARCHAR2(150),
  PAYMENT_STATUS VARCHAR2(30) DEFAULT 'PENDING',
  EMAIL_NOTIFICATION_SENT_AT TIMESTAMP
);

CREATE TABLE ORDER_ITEMS (
  ORDER_ID NUMBER REFERENCES CUSTOMER_ORDERS(ORDER_ID),
  BOOK_ID NUMBER REFERENCES BOOKS(BOOK_ID),
  QUANTITY NUMBER NOT NULL,
  UNIT_PRICE NUMBER(12,2) NOT NULL,
  PRIMARY KEY (ORDER_ID, BOOK_ID)
);
```

Task 2 : ABC Ltd Scenario

Studi kasus:

ABC Ltd merupakan perusahaan yang ingin menggunakan database untuk mengelola pemesanan barang, persediaan produk, pengiriman, faktur, dan pembayaran pelanggan.

Berdasarkan studi kasus tersebut, tabel-tabel yang dibutuhkan adalah:

No.	Nama Tabel	Field
1	ABC_Customers	customer_id (PK), customer_name, address, email, phone
2	ABC_Suppliers	supplier_id (PK), supplier_name, address, email, phone
3	ABC_Products	product_id (PK), product_name, description, unit_price, preferred_supplier_id (FK), quantity_in_stock, reorder_level
4	ABC_Orders	order_id (PK), customer_id (FK), order_date, status
5	ABC_Order_Items	order_item_id (PK), order_id (FK), product_id (FK), quantity_ordered, unit_price
6	ABC_Dispatches	dispatch_id (PK), order_id (FK), dispatch_date, status
7	ABC_Dispatch_Items	dispatch_item_id (PK), dispatch_id (FK), order_item_id (FK), quantity_dispatched, unit_price

8	ABC_Invoices	invoice_id (PK), dispatch_id (FK), invoice_date, due_date, total_amount, status
9	ABC_Back_Orders	back_order_id (PK), order_item_id (FK), quantity_backordered, status, created_at
10	ABC_Supplier_Orders	purchase_order_id (PK), supplier_id (FK), order_date, status
11	ABC_Supplier_Order_Items	purchase_order_item_id (PK), purchase_order_id (FK), product_id (FK), back_order_id (FK), quantity_ordered, unit_cost
12	ABC_Payments	payment_id (PK), customer_id (FK), payment_date, amount, payment_method, payment_reference
13	ABC_Payment_Allocations	payment_allocation_id (PK), payment_id (FK), invoice_id (FK), amount_applied

Penjelasan relasi antar tabel:

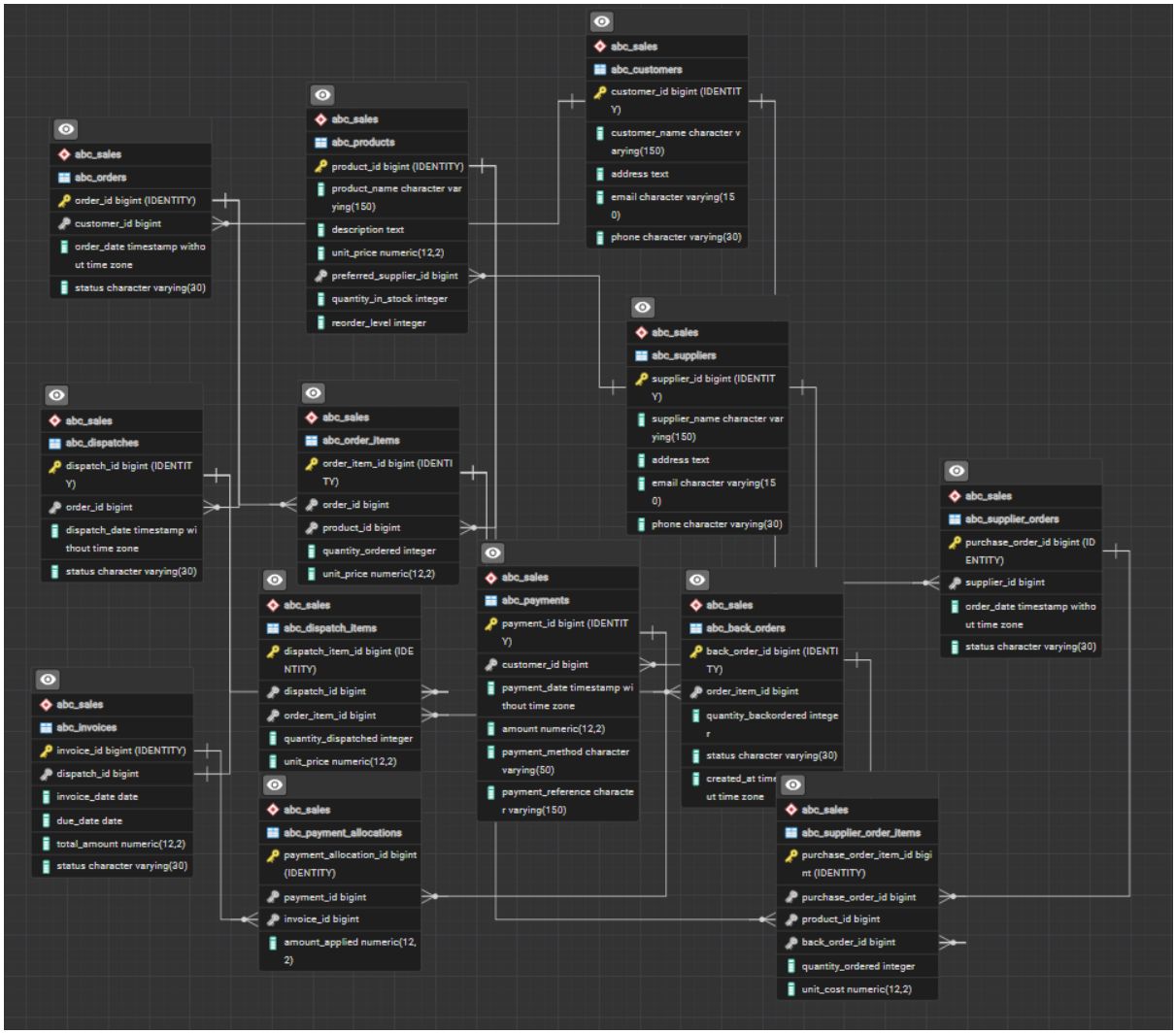
- Customers dengan Orders: satu pelanggan dapat membuat beberapa pesanan.
- Orders dengan Order_Items: satu pesanan dapat berisi beberapa produk.
- Products dengan Order_Items: satu produk dapat muncul di beberapa pesanan.
- Orders dengan Dispatches: satu pesanan dapat dikirim melalui beberapa pengiriman.
- Dispatches dengan Dispatch_Items: satu pengiriman dapat berisi beberapa produk.
- Dispatches dengan Invoices: satu pengiriman menghasilkan satu faktur.
- Order_Items dengan Back_Orders: barang yang belum tersedia akan dicatat sebagai pesanan tertunda.
- Suppliers dengan Products: setiap produk mempunyai pemasok utama.
- Suppliers dengan Supplier_Orders: perusahaan dapat melakukan pemesanan barang kepada beberapa pemasok.
- Customers dengan Payments: satu pelanggan dapat melakukan beberapa pembayaran.
- Payments dengan Invoices: hubungan pembayaran dan faktur menggunakan Payment_Allocations.

Kesimpulan Exercise 2:

Database ABC Ltd dirancang untuk mengelola proses pemesanan barang, persediaan, pengiriman, faktur, dan pembayaran.

Tabel Payment_Allocations sangat penting karena satu pembayaran dapat digunakan untuk membayar beberapa faktur, sedangkan satu faktur juga dapat dibayar melalui beberapa pembayaran.

ERD



Codingan oracle apex

```
CREATE TABLE ABC_CUSTOMERS (

    CUSTOMER_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,

    CUSTOMER_NAME VARCHAR2(150) NOT NULL,

    ADDRESS VARCHAR2(500),

    EMAIL VARCHAR2(150),

    PHONE VARCHAR2(30)

);



CREATE TABLE ABC_SUPPLIERS (

    SUPPLIER_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,

    SUPPLIER_NAME VARCHAR2(150) NOT NULL,
```

```
    ADDRESS VARCHAR2(500),
    EMAIL VARCHAR2(150),
    PHONE VARCHAR2(30)
);

CREATE TABLE ABC_PRODUCTS (
    PRODUCT_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    PRODUCT_NAME VARCHAR2(150) NOT NULL,
    DESCRIPTION VARCHAR2(500),
    UNIT_PRICE NUMBER(12,2) NOT NULL,
    PREFERRED_SUPPLIER_ID NUMBER REFERENCES ABC_SUPPLIERS(SUPPLIER_ID),
    QUANTITY_IN_STOCK NUMBER DEFAULT 0,
    REORDER_LEVEL NUMBER DEFAULT 0
);

CREATE TABLE ABC_ORDERS (
    ORDER_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    CUSTOMER_ID NUMBER REFERENCES ABC_CUSTOMERS(CUSTOMER_ID),
    ORDER_DATE TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    STATUS VARCHAR2(30) DEFAULT 'PENDING'
);

CREATE TABLE ABC_ORDER_ITEMS (
    ORDER_ITEM_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    ORDER_ID NUMBER REFERENCES ABC_ORDERS(ORDER_ID),
    PRODUCT_ID NUMBER REFERENCES ABC_PRODUCTS(PRODUCT_ID),
    QUANTITY_ORDERED NUMBER NOT NULL,
    UNIT_PRICE NUMBER(12,2) NOT NULL
);
```

```
CREATE TABLE ABC_DISPATCHES (
    DISPATCH_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    ORDER_ID NUMBER REFERENCES ABC_ORDERS(ORDER_ID),
    DISPATCH_DATE TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    STATUS VARCHAR2(30)
);

CREATE TABLE ABC_DISPATCH_ITEMS (
    DISPATCH_ITEM_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    DISPATCH_ID NUMBER REFERENCES ABC_DISPATCHES(DISPATCH_ID),
    ORDER_ITEM_ID NUMBER REFERENCES ABC_ORDER_ITEMS(ORDER_ITEM_ID),
    QUANTITY_DISPATCHED NUMBER NOT NULL,
    UNIT_PRICE NUMBER(12,2) NOT NULL
);

CREATE TABLE ABC_INVOICES (
    INVOICE_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    DISPATCH_ID NUMBER UNIQUE REFERENCES ABC_DISPATCHES(DISPATCH_ID),
    INVOICE_DATE DATE DEFAULT SYSDATE,
    DUE_DATE DATE,
    TOTAL_AMOUNT NUMBER(12,2) NOT NULL,
    STATUS VARCHAR2(30) DEFAULT 'UNPAID'
);

CREATE TABLE ABC_BACK_ORDERS (
    BACK_ORDER_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    ORDER_ITEM_ID NUMBER REFERENCES ABC_ORDER_ITEMS(ORDER_ITEM_ID),
    QUANTITY_BACKORDERED NUMBER NOT NULL,
    STATUS VARCHAR2(30) DEFAULT 'PENDING',
    CREATED_AT TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);


CREATE TABLE ABC_SUPPLIER_ORDERS (
    PURCHASE_ORDER_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    SUPPLIER_ID NUMBER REFERENCES ABC_SUPPLIERS(SUPPLIER_ID),
    ORDER_DATE TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    STATUS VARCHAR2(30) DEFAULT 'PENDING'
);


CREATE TABLE ABC_SUPPLIER_ORDER_ITEMS (
    PURCHASE_ORDER_ITEM_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    PURCHASE_ORDER_ID NUMBER REFERENCES ABC_SUPPLIER_ORDERS(PURCHASE_ORDER_ID),
    PRODUCT_ID NUMBER REFERENCES ABC_PRODUCTS(PRODUCT_ID),
    BACK_ORDER_ID NUMBER REFERENCES ABC_BACK_ORDERS(BACK_ORDER_ID),
    QUANTITY_ORDERED NUMBER NOT NULL,
    UNIT_COST NUMBER(12,2) NOT NULL
);


CREATE TABLE ABC_PAYMENTS (
    PAYMENT_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    CUSTOMER_ID NUMBER REFERENCES ABC_CUSTOMERS(CUSTOMER_ID),
    PAYMENT_DATE TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    AMOUNT NUMBER(12,2) NOT NULL,
    PAYMENT_METHOD VARCHAR2(50),
    PAYMENT_REFERENCE VARCHAR2(150)
);


CREATE TABLE ABC_PAYMENT_ALLOCATIONS (
    PAYMENT_ALLOCATION_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    PAYMENT_ID NUMBER REFERENCES ABC_PAYMENTS(PAYMENT_ID),
```

```
    INVOICE_ID NUMBER REFERENCES ABC_INVOICES(INVOICE_ID),

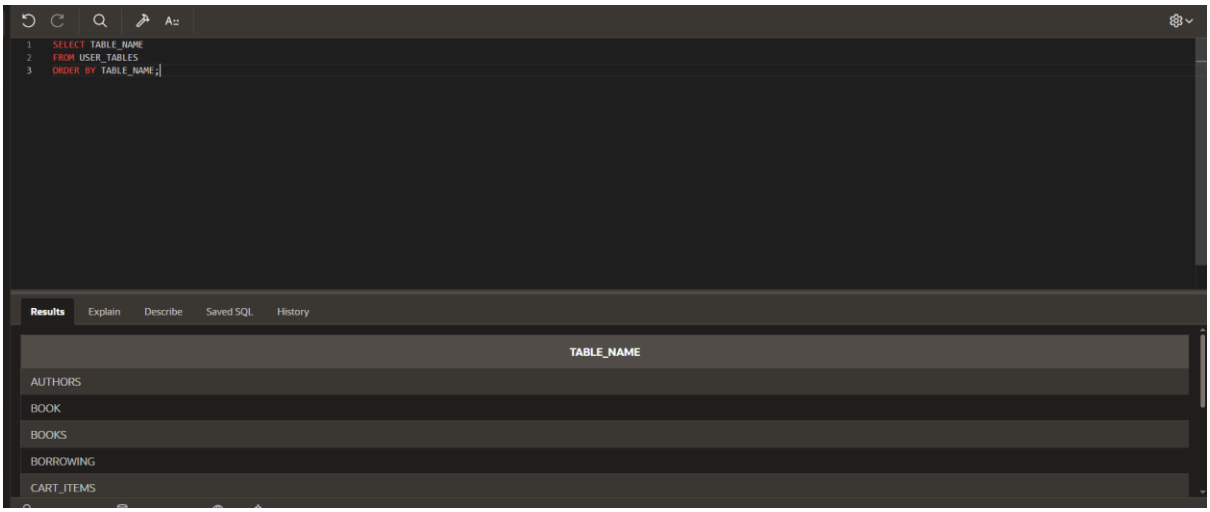
    AMOUNT_APPLIED NUMBER(12,2) NOT NULL

);
```

5. Kesimpulan

Melalui Section 2, dapat dipahami bahwa perancangan database dimulai dengan menentukan entity dan attribute, kemudian menentukan UID serta relationship antar-entity. Seluruh komponen tersebut selanjutnya digambarkan menggunakan ERD sebagai dasar dalam membangun struktur database yang terorganisasi.

Hasilnya:



Practice 2-2

Exercise 1: Conceptual and Physical Models

1. Provide five reasons for creating a conceptual data model.

Berikut lima alasan mengapa conceptual data model perlu dibuat:

1. Memahami kebutuhan bisnis
Membantu memahami jenis informasi dan data yang dibutuhkan oleh suatu organisasi atau sistem.

2. Mempermudah komunikasi
Membantu developer, database designer, dan pengguna memahami struktur data yang akan dibuat.

3. Mengidentifikasi entitas dan hubungan
Membantu menentukan entitas, atribut, serta hubungan antarentitas dalam sebuah sistem.

4. Mengurangi kesalahan desain
Membantu menemukan kekurangan atau kesalahan dalam perancangan database sebelum proses implementasi dilakukan.

5. Menjadi dasar pembuatan database
Digunakan sebagai acuan untuk mengembangkan logical data model dan physical data model.

2. List two examples of conceptual models and physical models.

Berikut dua contoh dari masing-masing model:

A. Conceptual Data Model

1. Entity Relationship Diagram (ERD)
Diagram yang menggambarkan entitas dan hubungan antarentitas dalam suatu sistem tanpa membahas detail teknis database.

2. Business Domain Model
Model yang menggambarkan objek atau entitas utama dalam proses bisnis beserta hubungan di antaranya.

B. Physical Data Model

1. Database Table Structure
Struktur tabel database yang sudah memiliki nama kolom, tipe data, Primary Key, dan Foreign Key.

2. SQL Database Schema
Implementasi database menggunakan perintah SQL, seperti CREATE TABLE, untuk membuat tabel dan hubungan antar tabel di PostgreSQL atau Oracle Database.

Kesimpulan

Conceptual Data Model digunakan untuk menggambarkan kebutuhan dan struktur data secara umum sebelum database dibuat. Sedangkan Physical Data Model digunakan untuk menggambarkan bagaimana struktur tersebut diterapkan secara teknis ke dalam database.

Kedua model tersebut saling berkaitan karena conceptual model menjadi dasar dalam proses pembuatan physical model.

Practice 2-3

Exercise 1: Identify and Draw Entities as a Beginning of an ERD

1. Tujuan Praktikum

Praktikum ini bertujuan untuk mengidentifikasi entitas yang dibutuhkan dalam Academic Database atau School Management System.

Entitas merupakan objek utama yang datanya akan disimpan di dalam database. Setiap entitas nantinya memiliki atribut yang menjelaskan informasi dari objek tersebut.

Exercise 2: Identify and Add Attributes and Corresponding Mandatory and Optional Notation to ERD

1. Pengertian Attribute

Attribute merupakan karakteristik atau informasi yang dimiliki oleh suatu entitas.

Contohnya, entitas STUDENT memiliki atribut berupa Student ID, Student Name, Email, dan Address.

Dalam praktikum ini terdapat dua jenis optionality notation:

- * Mandatory: atribut wajib diisi dan tidak boleh kosong.
- ° Optional: atribut boleh dikosongkan atau tidak wajib diisi.

Kesimpulan

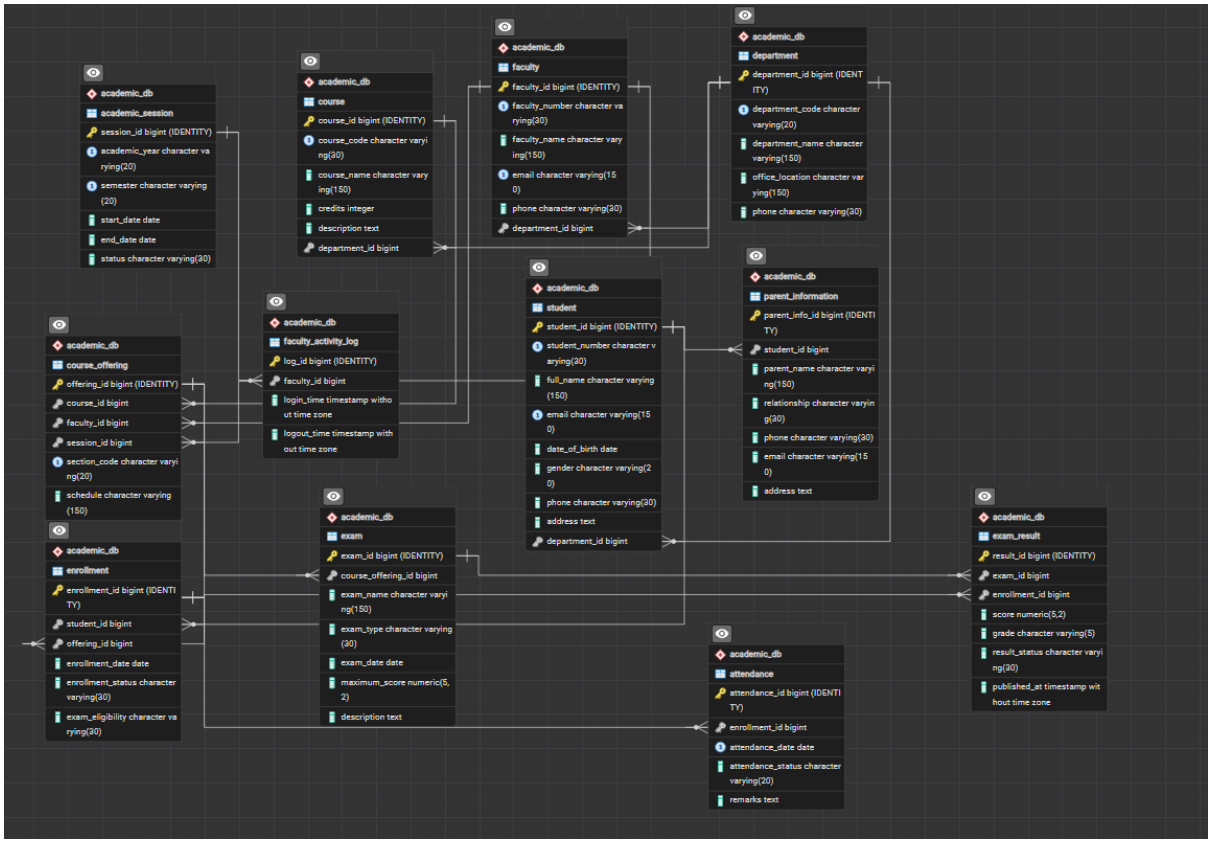
Berdasarkan praktikum ini, dapat dipahami bahwa entitas dan atribut merupakan bagian penting dalam perancangan database.

Entitas digunakan untuk menentukan objek yang akan disimpan, sedangkan atribut digunakan untuk menjelaskan informasi dari setiap entitas.

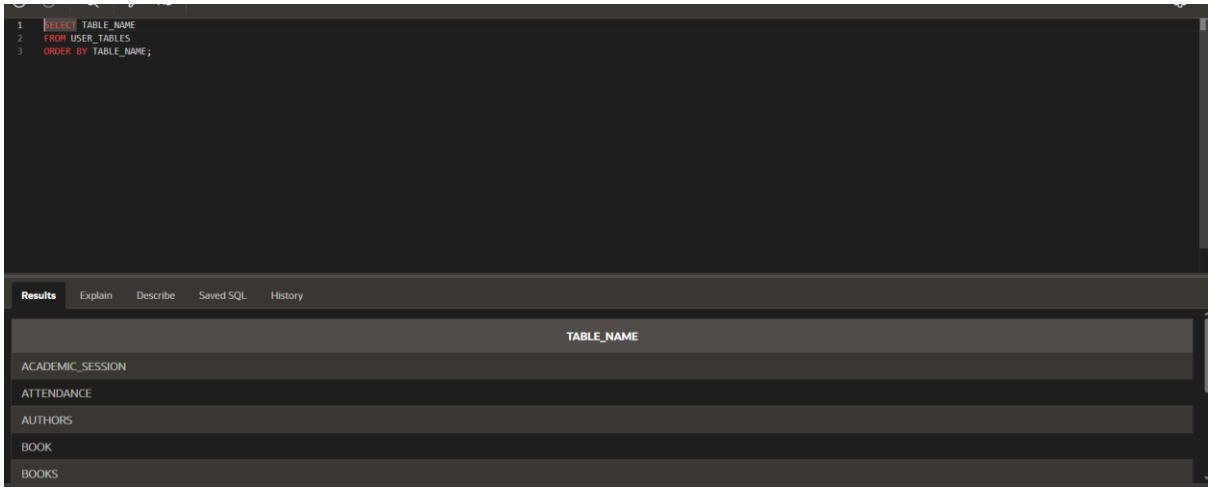
Selain itu, mandatory dan optional notation digunakan untuk menentukan atribut yang wajib diisi dan atribut yang boleh dikosongkan.

Dengan mengidentifikasi entitas, atribut, dan hubungannya, rancangan ERD dapat dibuat dengan lebih terstruktur sebelum diimplementasikan ke dalam database.

ERD



Hasilnya:



PRACTICE 2-4

Exercise 1: Identify the Unique Identifier and Corresponding Primary Keys

1. How do you find a particular song in the whole collection? What would be a unique identifier for SONG?

Untuk menemukan lagu tertentu dalam sebuah koleksi, kita dapat menggunakan Song ID sebagai Unique Identifier.

Song ID digunakan sebagai identitas unik setiap lagu sehingga tidak tertukar dengan lagu lainnya, meskipun memiliki judul yang sama.

Alternatifnya, kombinasi judul lagu, nama penyanyi, dan album juga dapat digunakan sebagai identitas unik apabila kombinasi tersebut dijamin tidak sama.

2. Which attribute or attributes allow you to pick a single student from the rest of the class?

Atribut yang dapat digunakan adalah Student ID atau NIM.

Setiap mahasiswa memiliki nomor identitas yang berbeda sehingga dapat digunakan untuk membedakan satu mahasiswa dengan mahasiswa lainnya.

Nama depan, nama belakang, dan alamat tidak selalu unik karena beberapa mahasiswa dapat memiliki informasi yang sama.

3. Select the attribute that could be the unique identifier of each entity.

A. Entity: STUDENT

Unique Identifier: Student ID

Student ID digunakan karena setiap mahasiswa memiliki nomor identitas yang berbeda.

B. Entity: MOVIE

Unique Identifier: Title + Date Released

Kombinasi judul film dan tanggal rilis digunakan untuk membedakan film yang memiliki judul sama tetapi tanggal rilis berbeda.

C. Entity: LOCKER

Unique Identifier: Location + Number

Kombinasi lokasi dan nomor locker digunakan untuk membedakan setiap locker, terutama jika nomor yang sama dapat digunakan di lokasi berbeda.

Exercise 2: Identify the Unique Identifiers and Add to the ERD

1. Identify the following:

a. Unique Identifiers

Unique Identifier merupakan atribut atau kombinasi atribut yang dapat membedakan setiap record dalam suatu entitas.

Berdasarkan Academic Database dari Exercise 2-3, berikut Unique Identifier yang digunakan:

1. DEPARTMENT: department_id
2. COURSE: course_id
3. STUDENT: student_id
4. FACULTY: faculty_id
5. ACADEMIC SESSION: session_id
6. PARENT INFORMATION: parent_info_id
7. EXAM: exam_id
8. COURSE OFFERING: offering_id
9. ENROLLMENT: enrollment_id
10. ATTENDANCE: attendance_id
11. EXAM RESULT: result_id
12. FACULTY ACTIVITY LOG: log_id

b. Candidate Unique Identifiers

Candidate Unique Identifier merupakan atribut atau kombinasi atribut lain yang juga dapat mengidentifikasi sebuah record secara unik dan dapat menjadi alternatif Primary Key.

Berdasarkan entitas Academic Database, kandidatnya adalah:

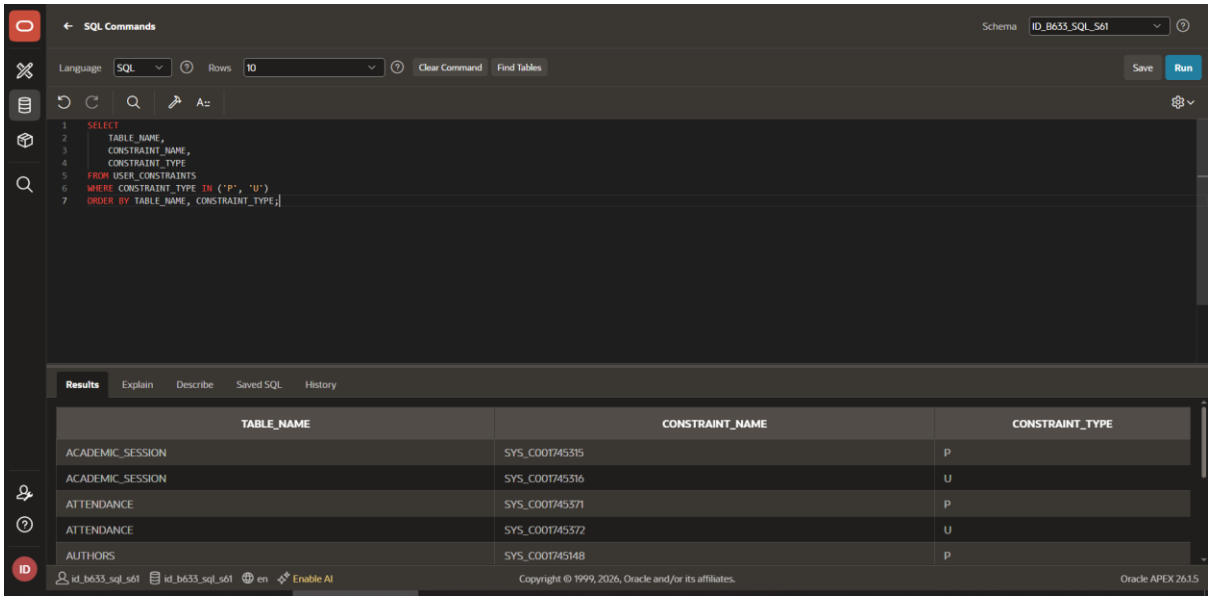
1. DEPARTMENT: department_code
2. COURSE: course_code
3. STUDENT: student_number, email
4. FACULTY: faculty_number, email
5. ACADEMIC SESSION: academic_year + semester
6. PARENT INFORMATION: Belum terdapat atribut lain yang dijamin unik.
7. EXAM: Belum terdapat atribut lain yang dijamin unik.
8. COURSE OFFERING: course_id + session_id + section_code
9. ENROLLMENT: student_id + offering_id
10. ATTENDANCE: enrollment_id + attendance_date
11. EXAM RESULT: exam_id + enrollment_id
12. FACULTY ACTIVITY LOG: Belum terdapat atribut lain yang dijamin unik.

Kesimpulan

Unique Identifier digunakan untuk membedakan setiap data dalam sebuah entitas. Primary Key merupakan Unique Identifier yang dipilih sebagai identitas utama, sedangkan Candidate Unique Identifier merupakan alternatif yang juga dapat mengidentifikasi data secara unik.

Dengan menentukan Unique Identifier, database dapat menghindari duplikasi identitas dan menjaga konsistensi data.

Hasil:



PRACTICE 2-5

Exercise 1: Identify Relationships from the ERD

1. Read the relationship. Which text corresponds to the diagram?

Jawaban: A.

Each EMPLOYEE may be assigned to one or more DEPARTMENTs. Each DEPARTMENT must be responsible for one or more EMPLOYEEs.

Penjelasan:

Jawaban A sesuai dengan diagram berdasarkan posisi simbol crow's foot (kaki gagak). Simbol tersebut menunjukkan hubungan dengan jumlah banyak (many).

Diagram menunjukkan bahwa seorang Employee dapat berhubungan dengan beberapa Department, sedangkan setiap Department harus memiliki satu atau lebih Employee yang menjadi tanggung jawabnya.

2. Read the relationship between PERSON and TOWN.

Relasi 1: Born in / Birthplace of

ERDish:

Each PERSON must be born in one or more TOWNs. Each TOWN must be the birthplace of one or more PERSONs.

Comment:

Berdasarkan diagram, seseorang dapat memiliki hubungan dengan beberapa kota tempat lahir. Namun, dalam kehidupan nyata, seseorang hanya lahir di satu kota, sedangkan satu kota dapat menjadi tempat lahir bagi banyak orang.

Relasi 2: Living in / Hometown of

ERDish:

Each PERSON may be living in one or more TOWNs. Each TOWN may be the hometown of one or more PERSONs.

Comment:

Seseorang dapat tinggal di beberapa kota sepanjang hidupnya. Satu kota juga dapat menjadi tempat tinggal bagi banyak orang. Namun, jika yang dicatat hanya tempat tinggal saat ini, seseorang biasanya memiliki satu kota tempat tinggal utama.

Relasi 3: Visitor of / Visited by

ERDish:

Each PERSON may be a visitor of one or more TOWNs. Each TOWN may be visited by one or more PERSONs.

Comment:

Seseorang dapat mengunjungi banyak kota, dan satu kota dapat dikunjungi oleh banyak orang. Hubungan ini sesuai dengan aktivitas perjalanan manusia.

Relasi 4: Mayor of / Governed by

ERDish:

Each PERSON may be the mayor of exactly one TOWN. Each TOWN must be governed by exactly one PERSON.

Comment:

Seseorang dapat menjadi walikota di satu kota, sedangkan setiap kota dalam model ini memiliki satu orang yang bertugas sebagai walikota.

Exercise 2: Analyze and Model Relationships

1. Write the ERDish for each relationship in the Academic Database.

Berdasarkan Academic Database yang dibuat pada Practice 2-3, berikut hubungan antarentitas beserta ERDish-nya.

1. DEPARTMENT dan COURSE (Offers)

Each COURSE must be offered by exactly one DEPARTMENT. Each DEPARTMENT may offer zero or more COURSEs.

Artinya, setiap mata kuliah harus berada di bawah satu departemen, sedangkan satu departemen dapat menawarkan beberapa mata kuliah.

2. DEPARTMENT dan FACULTY (Employs)

Each FACULTY must belong to exactly one DEPARTMENT. Each DEPARTMENT may employ zero or more FACULTYs.

Artinya, setiap dosen berada di satu departemen, dan satu departemen dapat memiliki beberapa dosen.

3. DEPARTMENT dan STUDENT (Manages)

Each STUDENT must belong to exactly one DEPARTMENT. Each DEPARTMENT may manage zero or more STUDENTs.

Artinya, setiap mahasiswa terdaftar pada satu departemen, sedangkan satu departemen dapat mengelola banyak mahasiswa.

4. STUDENT dan PARENT INFORMATION (Has)

Each PARENT INFORMATION must belong to exactly one STUDENT. Each STUDENT must have one or more PARENT INFORMATION records.

Artinya, setiap informasi orang tua atau wali terhubung dengan satu mahasiswa. Seorang mahasiswa dapat memiliki beberapa data orang tua atau wali.

5. COURSE dan COURSE OFFERING (Has Offering)

Each COURSE OFFERING must belong to exactly one COURSE. Each COURSE may have zero or more COURSE OFFERINGs.

Artinya, setiap penawaran mata kuliah berasal dari satu mata kuliah, sedangkan satu mata kuliah dapat ditawarkan pada beberapa periode akademik.

6. FACULTY dan COURSE OFFERING (Teaches)

Each COURSE OFFERING must be taught by exactly one FACULTY. Each FACULTY may teach zero or more COURSE OFFERINGs.

Artinya, satu penawaran mata kuliah diajar oleh satu dosen, sedangkan seorang dosen dapat mengajar beberapa kelas mata kuliah.

7. ACADEMIC SESSION dan COURSE OFFERING (Schedules)

Each COURSE OFFERING must belong to exactly one ACADEMIC SESSION. Each ACADEMIC SESSION may have zero or more COURSE OFFERINGs.

Artinya, setiap penawaran mata kuliah harus berada dalam satu periode akademik, seperti semester ganjil atau semester genap.

8. STUDENT dan ENROLLMENT (Registers)

Each ENROLLMENT must belong to exactly one STUDENT. Each STUDENT may have zero or more ENROLLMENTs.

Artinya, setiap pendaftaran mata kuliah dimiliki oleh satu mahasiswa, sedangkan mahasiswa dapat mendaftar beberapa mata kuliah.

9. COURSE OFFERING dan ENROLLMENT (Receives)

Each ENROLLMENT must belong to exactly one COURSE OFFERING. Each COURSE OFFERING may have zero or more ENROLLMENTs.

Artinya, setiap pendaftaran mengacu pada satu kelas mata kuliah, sedangkan satu kelas dapat menerima pendaftaran dari banyak mahasiswa.

10. ENROLLMENT dan ATTENDANCE (Records)

Each ATTENDANCE must belong to exactly one ENROLLMENT. Each ENROLLMENT may have zero or more ATTENDANCE records.

Artinya, setiap catatan kehadiran terhubung dengan satu pendaftaran mahasiswa, sedangkan satu pendaftaran dapat memiliki banyak catatan kehadiran.

11. COURSE OFFERING dan EXAM (Conducts)

Each EXAM must belong to exactly one COURSE OFFERING. Each COURSE OFFERING may have zero or more EXAMs.

Artinya, setiap ujian dilaksanakan untuk satu kelas mata kuliah, sedangkan satu kelas mata kuliah dapat memiliki beberapa ujian.

12. EXAM dan EXAM RESULT (Produces)

Each EXAM RESULT must belong to exactly one EXAM. Each EXAM may have zero or more EXAM RESULTs.

Artinya, setiap hasil ujian terhubung dengan satu ujian, sedangkan satu ujian dapat menghasilkan banyak nilai mahasiswa.

13. ENROLLMENT dan EXAM RESULT (Receives Result)

Each EXAM RESULT must belong to exactly one ENROLLMENT. Each ENROLLMENT may have zero or more EXAM RESULTs.

Artinya, setiap hasil ujian dimiliki oleh satu pendaftaran mahasiswa, sedangkan satu pendaftaran dapat memiliki beberapa hasil ujian.

14. FACULTY dan FACULTY ACTIVITY LOG (Has Log)

Each FACULTY ACTIVITY LOG must belong to exactly one FACULTY. Each FACULTY may have zero or more FACULTY ACTIVITY LOGs.

Artinya, setiap catatan aktivitas login dan logout dimiliki oleh satu dosen, sedangkan seorang dosen dapat memiliki banyak catatan aktivitas.

Kesimpulan

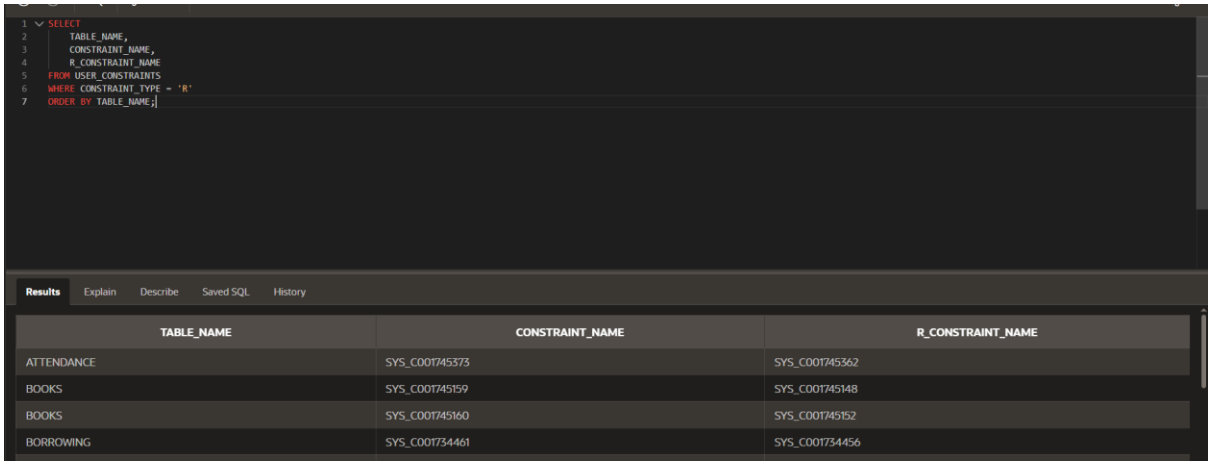
Berdasarkan Practice 2-5, dapat dipahami bahwa Relationship digunakan untuk menghubungkan satu entitas dengan entitas lainnya.

Hubungan tersebut dapat menunjukkan apakah suatu entitas berhubungan dengan satu data atau banyak data. ERDish membantu menjelaskan hubungan tersebut menggunakan kalimat yang mudah dipahami.

Dengan menentukan relationship, optionality, dan cardinality, struktur ERD menjadi lebih jelas dan dapat digunakan sebagai dasar dalam pembuatan database.

Karena codingannya sama

HASIL:



```
SELECT
    TABLE_NAME,
    CONSTRAINT_NAME,
    R_CONSTRAINT_NAME
FROM USER_CONSTRAINTS
WHERE CONSTRAINT_TYPE = 'R'
ORDER BY TABLE_NAME;
```

Results | Explain | Describe | Saved SQL | History

TABLE_NAME	CONSTRAINT_NAME	R_CONSTRAINT_NAME
ATTENDANCE	SYS_C001745373	SYS_C001745362
BOOKS	SYS_C001745159	SYS_C001745148
BOOKS	SYS_C001745160	SYS_C001745152
BORROWING	SYS_C001734461	SYS_C001734456

PRACTICE 2-6

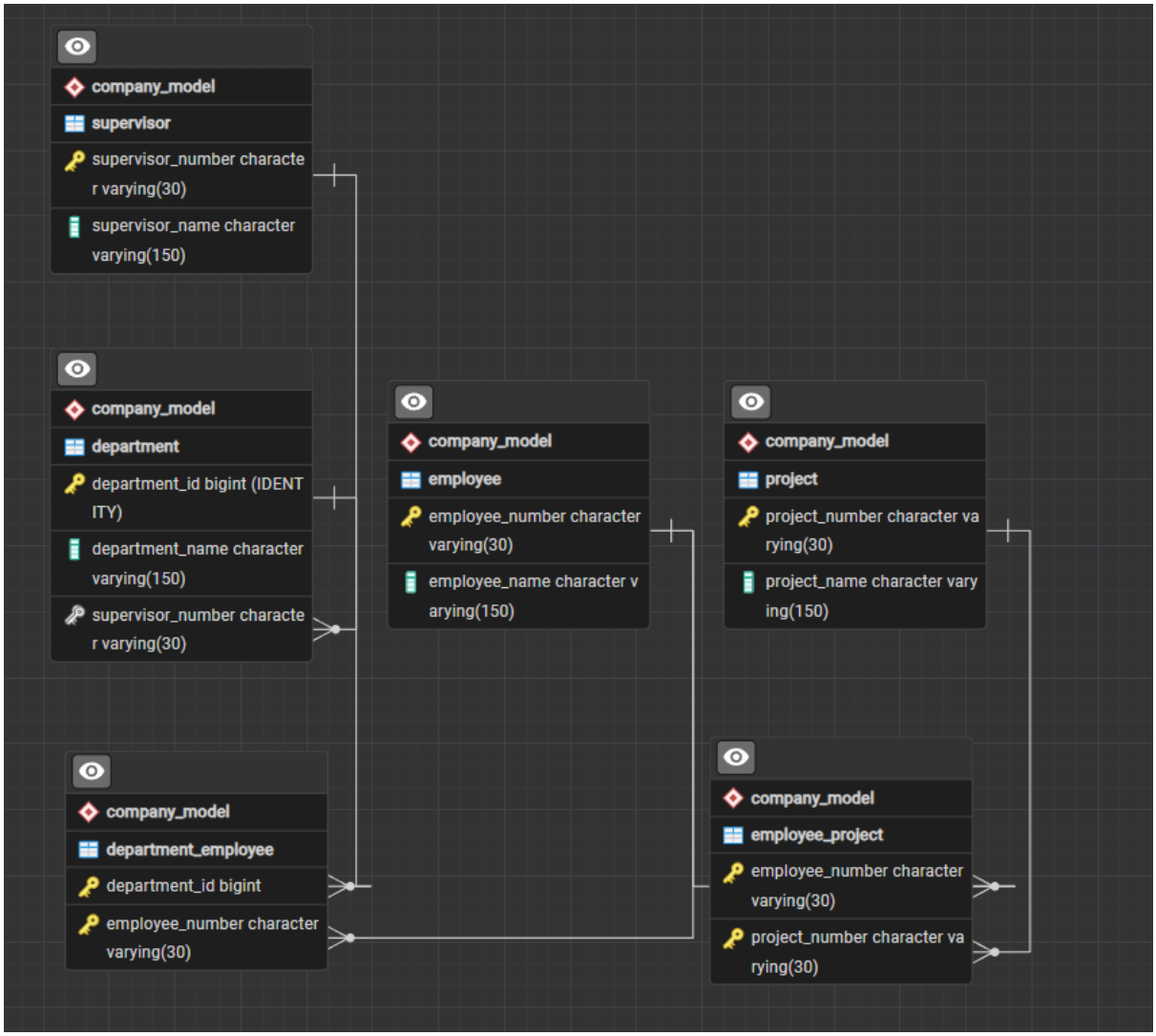
Exercise 1: Identify the Components in the ERD

1. Identify the Possible Entities and Attributes

Berdasarkan studi kasus perusahaan, terdapat empat entitas utama dan dua entitas penghubung. Company Database

Entity	Attributes
DEPARTMENT	# department_id, * department_name, * supervisor_number (FK)
SUPERVISOR	# supervisor_number, * supervisor_name
EMPLOYEE	# employee_number, * employee_name
PROJECT	# project_number, * project_name
DEPARTMENT_EMPLOYEE	# department_id (PK, FK), # employee_number (PK, FK)
EMPLOYEE_PROJECT	# employee_number (PK, FK), # project_number (PK, FK)

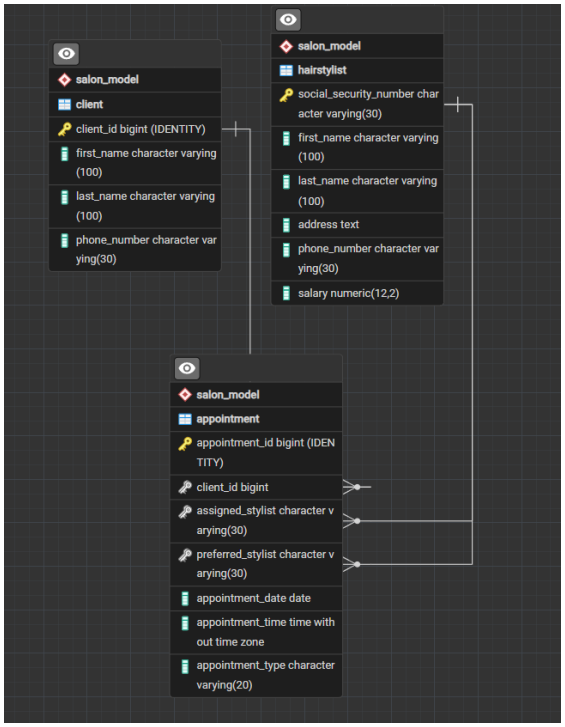
ERD



Exercise 2: Identify Entities, Attributes, Mandatory, Optional, and UID

Salon Database

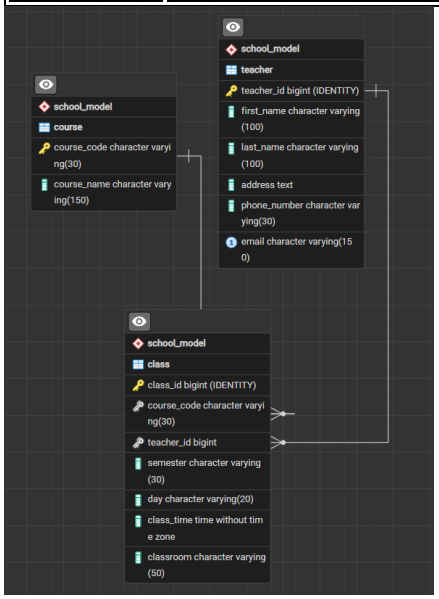
Entity	Attributes
HAIRSTYLIST	# social_security_number, * first_name, * last_name, * address, * phone_number, * salary
CLIENT	# client_id, * first_name, ° last_name, ° phone_number
APPOINTMENT	# appointment_id, * client_id (FK), * assigned_stylist (FK), ° preferred_stylist (FK), * appointment_date, * appointment_time, * appointment_type



Exercise 3: Identify Entities, Attributes, Mandatory, Optional, and UID

School Database:

Entity	Attributes
TEACHER	# teacher_id, * first_name, * last_name, * address, * phone_number, * email
COURSE	# course_code, * course_name
CLASS	# class_id, * course_code (FK), * teacher_id (FK), * semester, * day, * class_time, * classroom



Kesimpulan

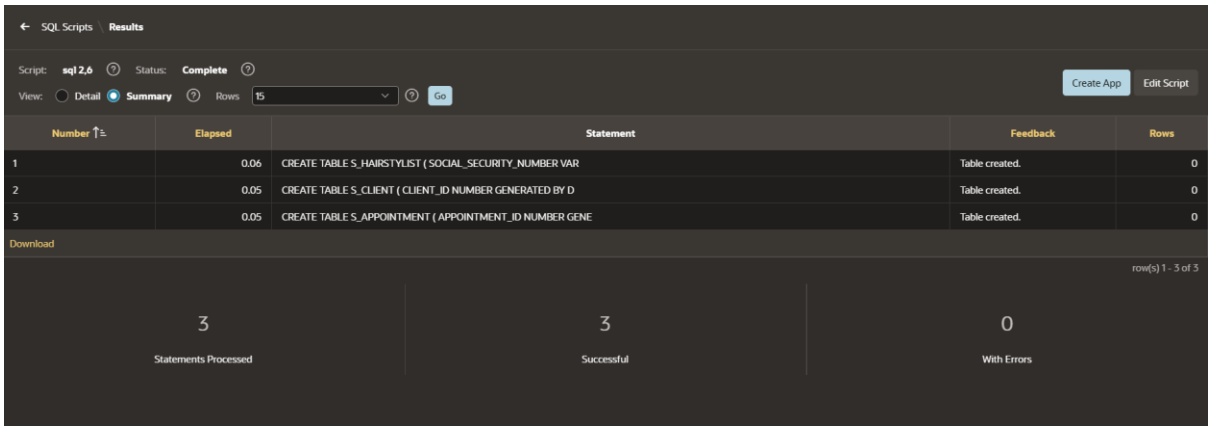
Berdasarkan praktikum 2-6, dapat dipahami bahwa Entity Relationship Diagram digunakan untuk menggambarkan entitas, atribut, dan hubungan antarentitas dalam sebuah sistem.

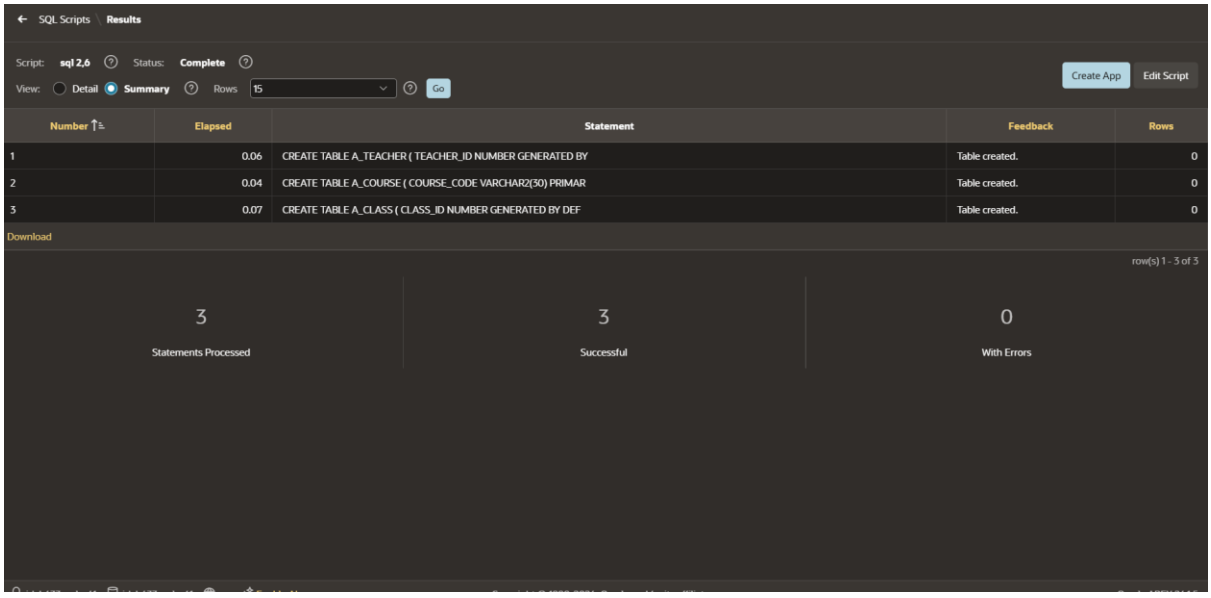
Setiap entitas memiliki Unique Identifier sebagai identitas unik. Atribut mandatory wajib diisi, sedangkan atribut optional boleh dikosongkan.

Selain itu, entitas penghubung dapat digunakan untuk mengelola hubungan antarentitas, seperti penugasan karyawan pada proyek, jadwal kunjungan pelanggan, dan penempatan guru pada kelas.

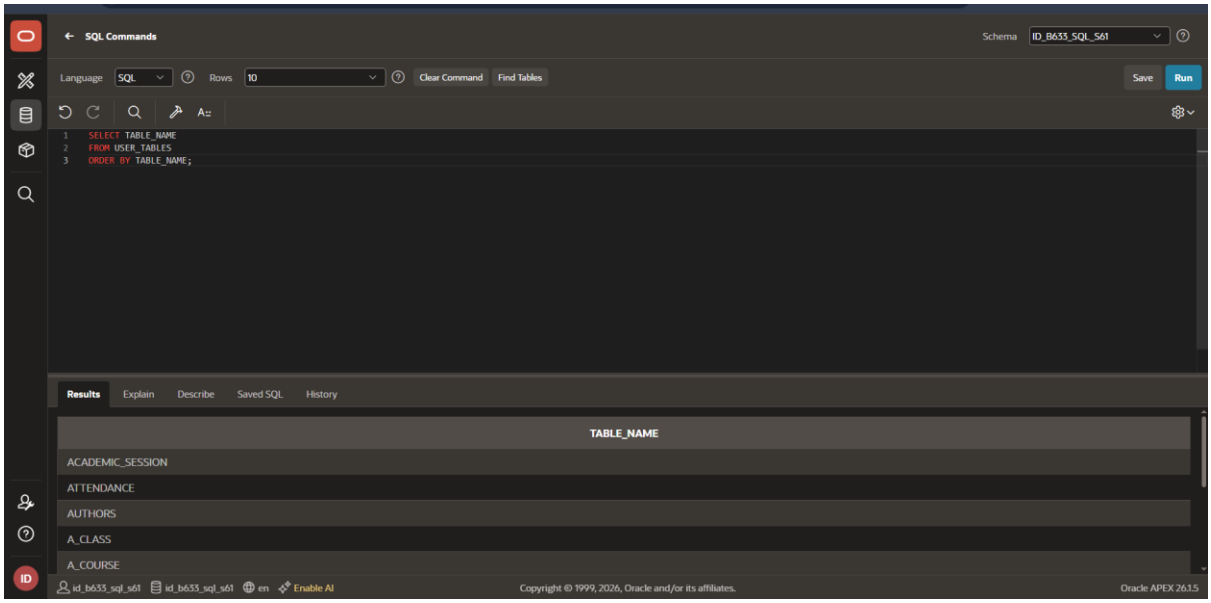
Dengan ERD, struktur database dapat dirancang lebih jelas sebelum diterapkan menggunakan SQL.

Input:





Output:



4. SECTION-3

Exercise 1: Resolve M:M Relationships

1. Resolve M:M between STUDENT and COURSE

Hubungan Many-to-Many antara STUDENT dan COURSE diselesaikan menggunakan entitas penghubung bernama STUDENT_COURSE.

Entity	Attributes
STUDENT_COURSE	# enrollment_id, * student_id (FK), * course_id (FK), * session_id (FK), * enrollment_date, ° grade

Setiap mahasiswa dapat mengambil beberapa mata kuliah, dan setiap mata kuliah dapat diikuti oleh banyak mahasiswa.

2. Resolve M:M between FACULTY and COURSE

Hubungan Many-to-Many antara FACULTY dan COURSE diselesaikan menggunakan entitas penghubung bernama FACULTY_COURSE.

Entity	Attributes
FACULTY_COURSE	# faculty_course_id, * faculty_id (FK), * course_id (FK), * session_id (FK), * contact_hours

Entitas ini digunakan untuk mencatat dosen yang mengajar mata kuliah tertentu dalam suatu periode akademik.

3. Resolve M:M between STUDENT, COURSE, and EXAM

Hubungan tersebut diselesaikan menggunakan tiga entitas, yaitu STUDENT_COURSE, EXAM, dan EXAM_RESULT.

Entity	Attributes
EXAM	# exam_id, * course_id (FK), * session_id (FK), * exam_name, ° exam_date
EXAM_RESULT	# result_id, * enrollment_id (FK), * exam_id (FK), ° grade, ° published_at

EXAM_RESULT digunakan untuk menghubungkan pendaftaran mahasiswa dengan ujian yang diikuti. Dengan demikian, nilai setiap mahasiswa dapat dicatat berdasarkan ujian pada mata kuliah tertentu.

Exercise 2: Adding Nontransferability Option to an ERD

Entity yang digunakan:

Entity	Attributes
STUDENT	# student_id, * first_name, * last_name, * email
EXAM_RESULT	# result_id, * student_id (FK), * exam_id (FK), ° grade, ° result_date

Relationship:

STUDENT memiliki EXAM_RESULT, sedangkan setiap EXAM_RESULT hanya dimiliki oleh satu STUDENT.

Nontransferability:

Hubungan antara STUDENT dan EXAM_RESULT bersifat non-transferable. Artinya, setelah hasil ujian diberikan kepada seorang mahasiswa, hasil tersebut tidak dapat dipindahkan atau dialihkan ke mahasiswa lain.

Exercise 3: Identify and Draw Supertype and Subtype Entities

FACULTY dijadikan sebagai supertype karena memiliki dua jenis dosen, yaitu Full Time dan Part Time.

Entity	Attributes
FACULTY (Supertype)	# faculty_id, * first_name, * last_name, * email
FULL_TIME (Subtype)	# faculty_id (PK, FK), * salary, * insurance
PART_TIME (Subtype)	# faculty_id (PK, FK), * hourly_rate

FACULTY menjadi entitas induk, sedangkan FULL_TIME dan PART_TIME menjadi entitas turunan.

Setiap dosen termasuk salah satu jenis, yaitu dosen tetap atau dosen paruh waktu. Kedua subtype tersebut memiliki atribut tambahan yang berbeda.

Exercise 4: Examine Exclusive Relationships (Arcs)

Entity yang digunakan:

Entity	Attributes
COURSE	# course_id, * course_name, * delivery_type
ONLINE	# course_id (PK, FK), * logon_id, * password_hash
SEATED	# course_id (PK, FK), * building, * room_number, * schedule

Exclusive Relationship (Arc):

Setiap COURSE hanya dapat diselenggarakan melalui salah satu pilihan, yaitu ONLINE atau SEATED.

- ONLINE: Mata kuliah dilakukan secara daring dan membutuhkan logon ID serta password.
- SEATED: Mata kuliah dilaksanakan di lokasi fisik dengan informasi gedung, ruangan, dan jadwal.

Kedua hubungan tersebut bersifat exclusive karena satu mata kuliah tidak dapat memiliki kedua jenis penyelenggaraan sekaligus.

Exercise 5: Model Hierarchical Data

Berdasarkan studi kasus hotel, terdapat empat entitas utama.

Entity	Attributes
HOTEL	# hotel_id, * hotel_name, * address
FLOOR	# floor_id, * hotel_id (FK), * floor_number
SUITE	# suite_id, * floor_id (FK), * suite_number
ROOM	# room_id, * suite_id (FK), * room_number

Struktur hierarki hotel dimulai dari HOTEL, kemudian memiliki beberapa FLOOR. Setiap FLOOR memiliki beberapa SUITE, dan setiap SUITE memiliki beberapa ROOM.

Exercise 6: Model Hierarchical Data and Recursive Relationships

Studi kasus Curves Dynamics memiliki struktur wilayah penjualan yang terdiri dari Region, District, Territory, dan Sales Area.

1. Hierarchical Structure

Struktur hierarki dibuat menggunakan entitas yang berbeda untuk setiap tingkatan.

Entity	Attributes
SALES EMPLOYEE	# employee_id, * employee_name, * employee_role
SALES DIRECTOR	# employee_id (PK, FK)
SALES MANAGER	# employee_id (PK, FK)
SALESPERSON	# employee_id (PK, FK), * sales_quota
SALES REGION	# region_code, * region_name, * director_id (FK)
SALES DISTRICT	# district_code, * district_name, * region_code (FK), * manager_id (FK)
SALES TERRITORY	# territory_code, * territory_name, * district_code (FK)
SALES AREA	# sales_area_code, * sales_area_name, * territory_code (FK), * salesperson_id (FK)

Struktur hierarkinya:

SALES REGION

|

SALES DISTRICT

|

SALES TERRITORY

|

SALES AREA

Setiap region dikelola oleh sales director, setiap district dikelola oleh sales manager, dan setiap sales area menjadi tanggung jawab satu salesperson.

2. Recursive Structure

Pada model recursive, seluruh tingkatan wilayah penjualan digabungkan ke dalam satu entitas bernama SALES UNIT. Entitas tersebut memiliki hubungan dengan dirinya sendiri melalui parent_unit_id.

Entity	Attributes
SALES UNIT	# unit_id, * unit_code, * unit_name, * unit_type, ° parent_unit_id (FK)
SALES EMPLOYEE	# employee_id, * employee_name, * employee_role
SALES RESPONSIBILITY	# responsibility_id, * employee_id (FK), * unit_id (FK), ° sales_quota

Struktur recursive:

```
SALES UNIT
  |
  +-- REGION
      |
      +-- DISTRICT
          |
          +-- TERRITORY
              |
              +-- SALES AREA
```

Pada model ini, parent_unit_id digunakan untuk menghubungkan suatu wilayah dengan wilayah induknya.

Perbedaan kedua model:

Hierarchical menggunakan tabel terpisah untuk setiap tingkatan wilayah, sedangkan Recursive menggunakan satu tabel SALES UNIT untuk menyimpan seluruh tingkatan wilayah dengan hubungan parent dan child.

Exercise 7: Developing a Complete ERD Using Supertype/Subtypes and Arcs

Berdasarkan studi kasus Right-Way Rental Truck Company, terdapat beberapa entitas utama untuk mengelola kantor rental, kendaraan, pelanggan, dan transaksi penyewaan.

1. Identifikasi Entity dan Attributes

Entity	Attributes
RENTAL OFFICE	# office_number, * office_name, * address, * is_central_office
VEHICLE	# vehicle_id, * registration_state, * license_plate, * vehicle_type_code (FK), * home_office_number (FK), * current_office_number (FK), * last_maintenance_date, * registration_expiration_date
VEHICLE TYPE	# type_code, * type_name, * vehicle_category
TRUCK (Subtype)	# vehicle_id (PK, FK), * current_odometer, * gas_tank_capacity, * has_working_radio
TRAILER (Subtype)	# vehicle_id (PK, FK), * maximum_weight_capacity
CUSTOMER (Supertype)	# customer_id, * customer_type
INDIVIDUAL CUSTOMER (Subtype)	# customer_id (PK, FK), * full_name, * home_phone, * address, * license_state, * license_number, * license_expiration, * poor_risk_status
COMPANY CUSTOMER (Subtype)	# customer_id (PK, FK), * company_number, * company_name, * address
RENTAL AGREEMENT	# office_number (PK, FK), # agreement_number (PK), * customer_id (FK), * vehicle_id (FK), * rental_date, * anticipated_duration, * dropoff_office_number (FK), * deposit_paid, * quoted_daily_rate, ° quoted_rate_per_mile, ° odometer_out, ° odometer_in
VEHICLE TRANSFER	# transfer_id, * vehicle_id (FK), * from_office_number (FK), * to_office_number (FK), * transfer_date

2. Supertype dan Subtype

VEHICLE menjadi supertype dengan dua subtype, yaitu TRUCK dan TRAILER.

- TRUCK menyimpan informasi odometer, kapasitas tangki bensin, dan kondisi radio.
- TRAILER menyimpan informasi kapasitas berat maksimal.

CUSTOMER menjadi supertype dengan dua subtype, yaitu INDIVIDUAL CUSTOMER dan COMPANY CUSTOMER.

- INDIVIDUAL CUSTOMER menyimpan identitas pribadi, informasi SIM, dan status pelanggan berisiko.
- COMPANY CUSTOMER menyimpan nomor perusahaan, nama, dan alamat perusahaan.

3. Exclusive Relationship (Arc)

Pada CUSTOMER terdapat dua kemungkinan jenis pelanggan, yaitu INDIVIDUAL CUSTOMER atau COMPANY CUSTOMER.

Hubungan tersebut bersifat exclusive karena satu rental agreement hanya boleh dimiliki oleh satu individu atau satu perusahaan, bukan keduanya sekaligus.

4. Hubungan Antarentitas

- RENTAL OFFICE mengelola banyak VEHICLE.
- VEHICLE memiliki satu VEHICLE TYPE.
- VEHICLE dibedakan menjadi TRUCK atau TRAILER.
- CUSTOMER dapat memiliki beberapa RENTAL AGREEMENT.
- Setiap RENTAL AGREEMENT hanya dimiliki oleh satu CUSTOMER.
- Setiap RENTAL AGREEMENT berlaku untuk satu VEHICLE.
- RENTAL OFFICE menjadi kantor asal dan kantor pengembalian kendaraan.
- VEHICLE TRANSFER mencatat perpindahan kendaraan antar kantor rental.

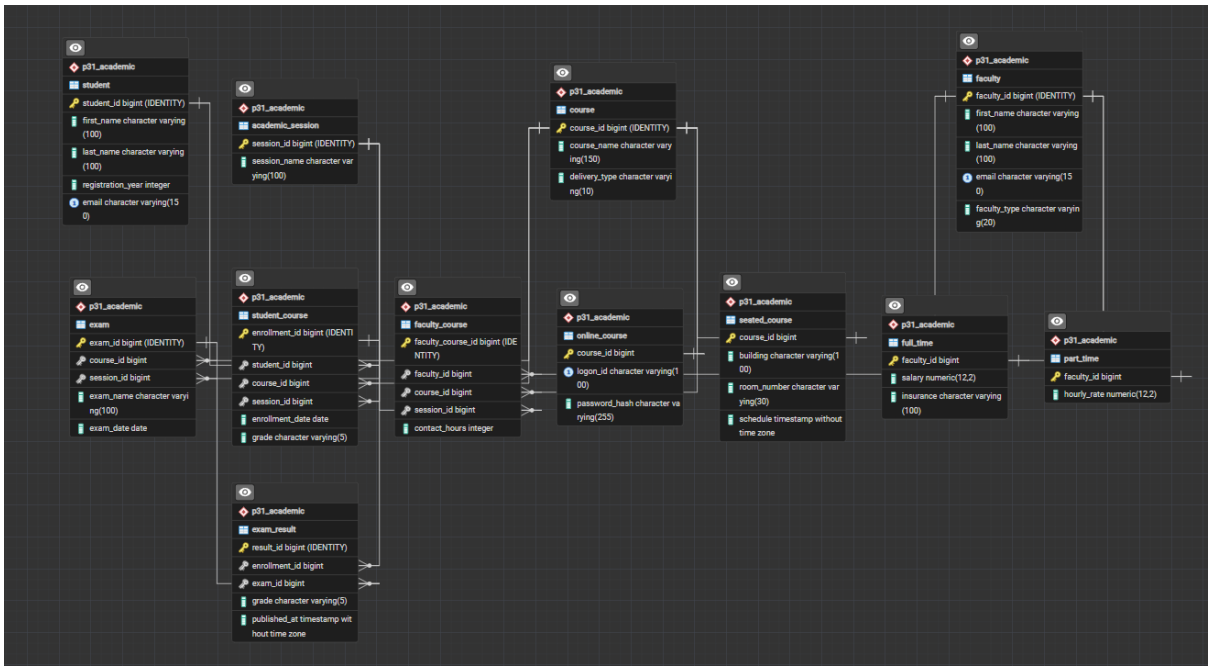
Sistem tidak menggunakan tabel reservation karena perusahaan tidak menerima pemesanan kendaraan sebelumnya.

Kesimpulan

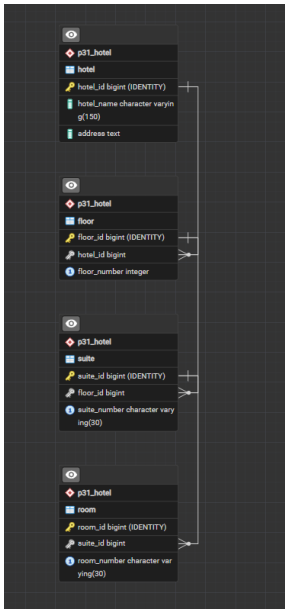
Berdasarkan Practice 3-1, dapat dipahami bahwa hubungan Many-to-Many perlu diselesaikan menggunakan intersection entity. Selain itu, konsep nontransferability, supertype dan subtype, exclusive relationship (arc), serta hierarchical dan recursive relationships membantu menghasilkan struktur ERD yang lebih lengkap.

Konsep-konsep tersebut dapat diterapkan untuk berbagai kebutuhan database, mulai dari sistem akademik, manajemen hotel, penjualan, hingga penyewaan kendaraan.

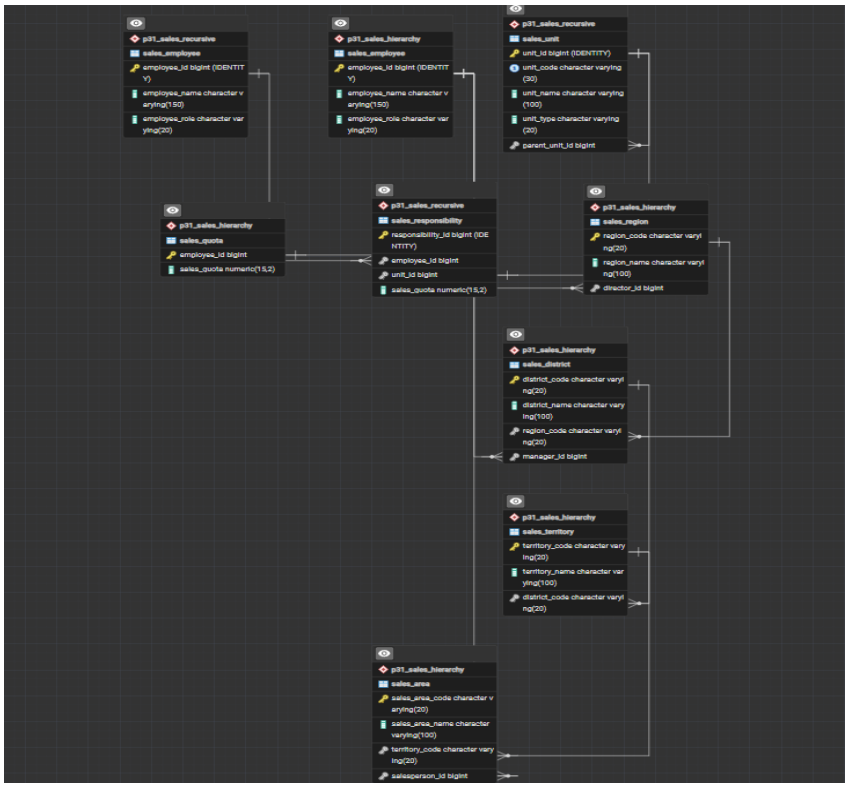
ERD EXERCISE 1-4



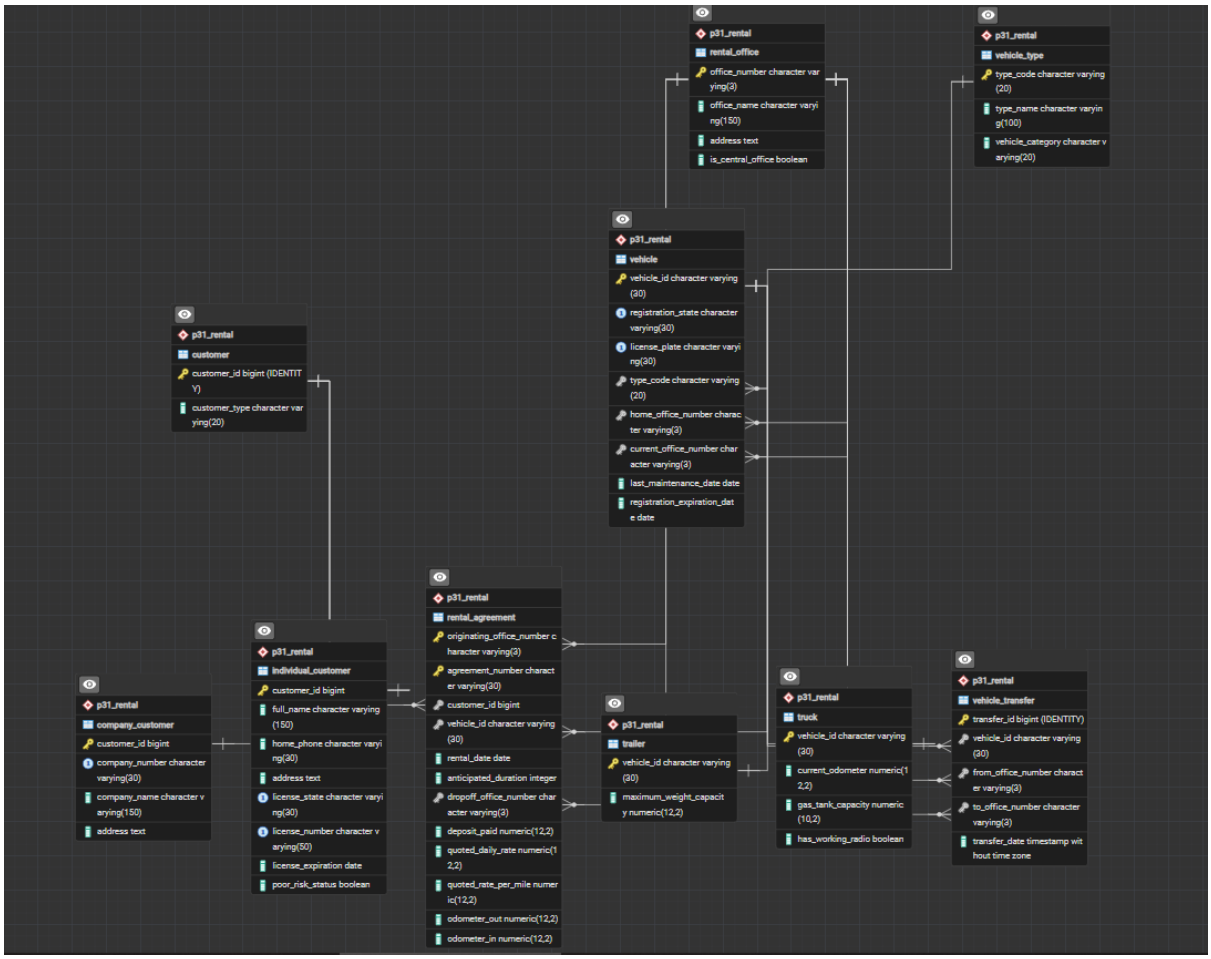
ERD EXERCISE 5



ERD EXERCISE 6

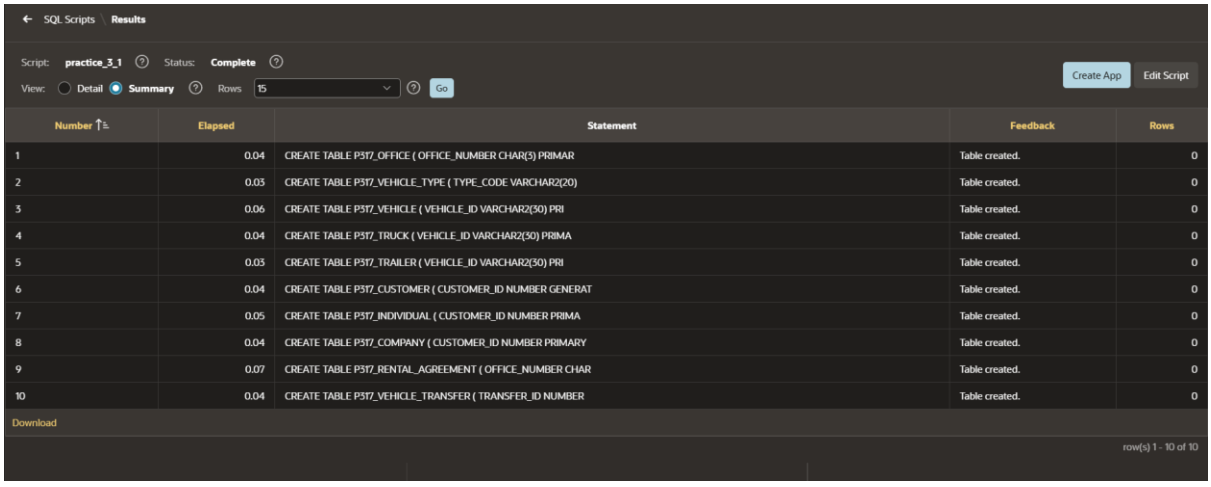


ERD EXERCISE 7



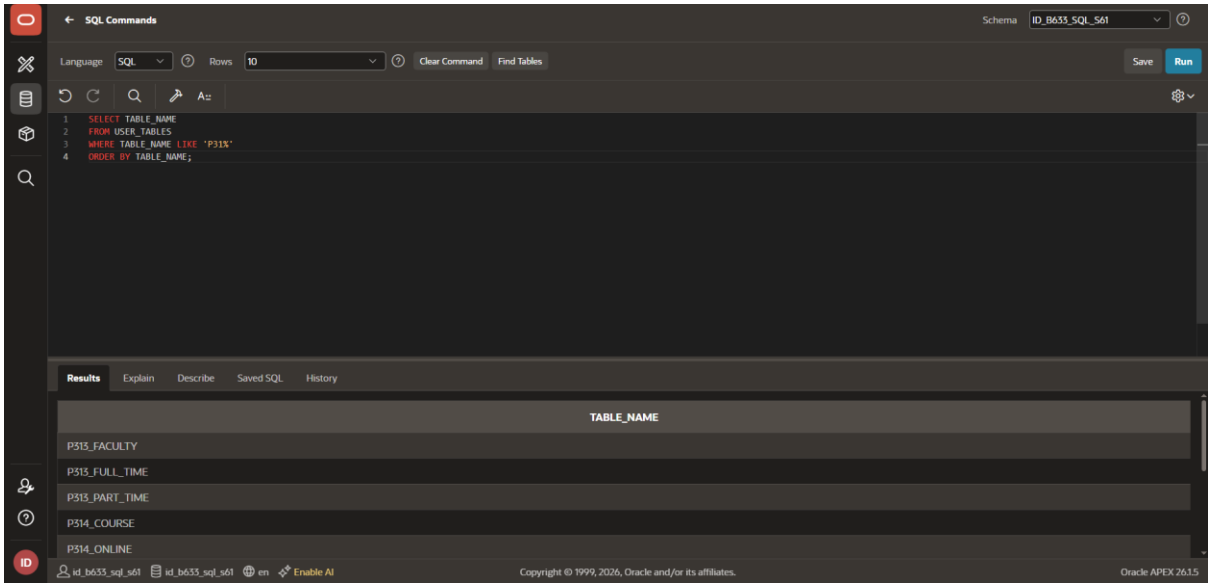
Pengujian pada Oracle Apex

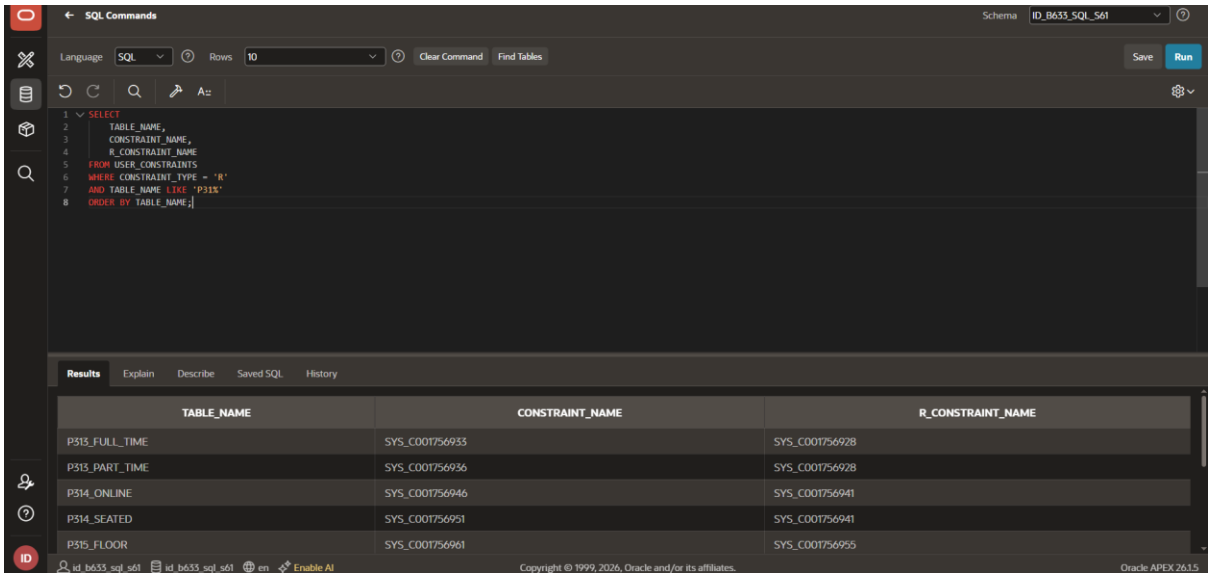
Salah satu contoh

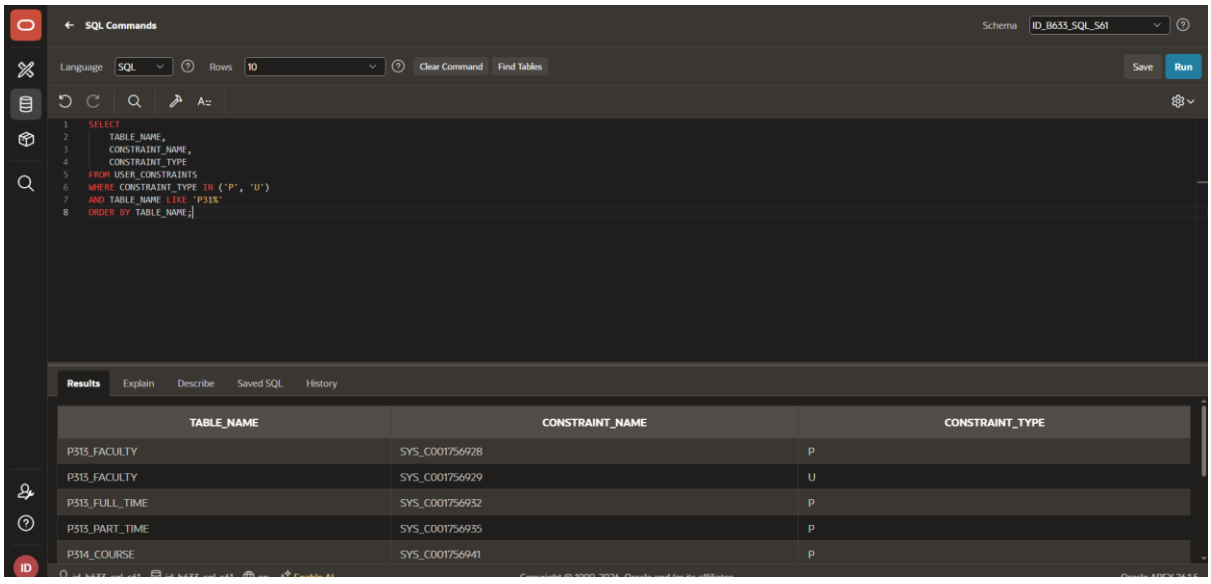


Number	Elapsed	Statement	Feedback	Rows
1	0.04	CREATE TABLE P317_OFFICE (OFFICE_NUMBER CHAR(3) PRIMAR	Table created.	0
2	0.03	CREATE TABLE P317_VEHICLE_TYPE (TYPE_CODE VARCHAR2(20)	Table created.	0
3	0.06	CREATE TABLE P317_VEHICLE (VEHICLE_ID VARCHAR2(30) PRI	Table created.	0
4	0.04	CREATE TABLE P317_TRUCK (VEHICLE_ID VARCHAR2(30) PRIMA	Table created.	0
5	0.03	CREATE TABLE P317_TRAILER (VEHICLE_ID VARCHAR2(30) PRI	Table created.	0
6	0.04	CREATE TABLE P317_CUSTOMER (CUSTOMER_ID NUMBER GENERAT	Table created.	0
7	0.05	CREATE TABLE P317_INDIVIDUAL (CUSTOMER_ID NUMBER PRIMA	Table created.	0
8	0.04	CREATE TABLE P317_COMPANY (CUSTOMER_ID NUMBER PRIMARY	Table created.	0
9	0.07	CREATE TABLE P317_RENTAL_AGREEMENT (OFFICE_NUMBER CHAR	Table created.	0
10	0.04	CREATE TABLE P317_VEHICLE_TRANSFER (TRANSFER_ID NUMBER	Table created.	0

Hasil (output):


SQL Commands
Schema ID_B633_SQL_S61
Language SQL
Rows 10
Clear Command
Find Tables
Save
Run
SELECT TABLE_NAME
FROM USER_TABLES
WHERE TABLE_NAME LIKE 'P31%'
ORDER BY TABLE_NAME;
Results
Explain
Describe
Saved SQL
History
TABLE_NAME
P313_FACULTY
P313_FULL_TIME
P313_PART_TIME
P314_COURSE
P314_ONLINE
Enable AI
Copyright © 1999, 2026, Oracle and/or its affiliates.
Oracle APEX 26.1.5


SQL Commands
Schema ID_B633_SQL_S61
Language SQL
Rows 10
Clear Command
Find Tables
Save
Run
SELECT
TABLE_NAME,
CONSTRAINT_NAME,
R_CONSTRAINT_NAME
FROM USER_CONSTRAINTS
WHERE CONSTRAINT_TYPE = 'R'
AND TABLE_NAME LIKE 'P31%'
ORDER BY TABLE_NAME;
Results
Explain
Describe
Saved SQL
History
TABLE_NAME
CONSTRAINT_NAME
R_CONSTRAINT_NAME
P313_FULL_TIME
SYS_C001756933
SYS_C001756928
P313_PART_TIME
SYS_C001756936
SYS_C001756928
P314_ONLINE
SYS_C001756946
SYS_C001756941
P314_SEATED
SYS_C001756951
SYS_C001756941
P315_FLOOR
SYS_C001756961
SYS_C001756955
Enable AI
Copyright © 1999, 2026, Oracle and/or its affiliates.
Oracle APEX 26.1.5


SQL Commands
Schema ID_B633_SQL_S61
Language SQL
Rows 10
Clear Command
Find Tables
Save
Run
SELECT
TABLE_NAME,
CONSTRAINT_NAME,
CONSTRAINT_TYPE
FROM USER_CONSTRAINTS
WHERE CONSTRAINT_TYPE IN ('P', 'U')
AND TABLE_NAME LIKE 'P31%'
ORDER BY TABLE_NAME;
Results
Explain
Describe
Saved SQL
History
TABLE_NAME
CONSTRAINT_NAME
CONSTRAINT_TYPE
P313_FACULTY
SYS_C001756928
P
P313_FACULTY
SYS_C001756929
U
P313_FULL_TIME
SYS_C001756932
P
P313_PART_TIME
SYS_C001756935
P
P314_COURSE
SYS_C001756941
P

PRACTICE 3-2

Exercise 1: Track Data Change over Time

1. Historical Grades

Untuk menyimpan riwayat nilai mahasiswa yang mengambil kembali mata kuliah, perlu ditambahkan entitas baru bernama STUDENT GRADE HISTORY.

Atribut entitas tersebut:

- `# Course Start Date`
- `* Course End Date`
- `* Previous Grade`

Hubungan antara STUDENT COURSE DETAIL dan STUDENT GRADE HISTORY adalah satu STUDENT COURSE DETAIL dapat memiliki beberapa catatan nilai sebelumnya.

Unique Identifier STUDENT GRADE HISTORY berasal dari UID STUDENT COURSE DETAIL yang diwariskan melalui barred relationship, ditambah Course Start Date.

Course End Date dan Previous Grade bersifat mandatory karena setiap riwayat mencatat nilai dari mata kuliah yang sudah diselesaikan.

Course Hero

Catatan: Sesuai instruksi soal, perubahan ini hanya ditampilkan sebagai rancangan dan tidak diterapkan ke ERD utama.

Exercise 2: Classroom Assignments for Different Exams

A. Why is Start Time part of the UID of ASSIGNMENT?

Start Time menjadi bagian dari Unique Identifier karena satu tanggal ujian dapat memiliki beberapa jadwal ujian dengan waktu yang berbeda.

Kombinasi Date of Exam dan Start Time digunakan untuk membedakan setiap jadwal assignment agar tidak terjadi duplikasi identitas.

B. Three Time-Related Constraints

1. End Time must be later than Start Time.

Waktu selesai ujian harus lebih lambat daripada waktu mulai ujian.

Not Conditional Non-transferability

2. Exam assignments in the same classroom must not overlap.

Satu ruangan tidak boleh digunakan untuk dua ujian pada waktu yang bersamaan di tanggal yang sama.

Not Conditional Non-transferability

3. An assignment cannot be transferred after it has started.

Assignment boleh dipindahkan ke ruangan lain atau dijadwalkan ulang sebelum ujian dimulai. Namun, setelah ujian dimulai, perubahan tersebut tidak diperbolehkan.

Conditional Non-transferability

Aturan ini sesuai konsep conditional non-transferability, yaitu hubungan masih boleh diubah sebelum kondisi tertentu terjadi, tetapi tidak boleh diubah setelah kondisi tersebut terpenuhi.

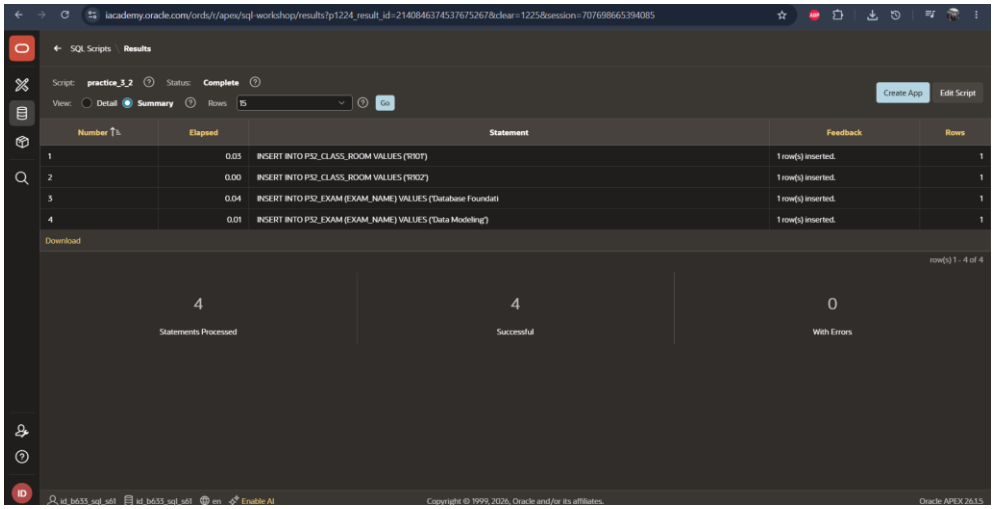
cs.utahtech.edu

Kesimpulan:

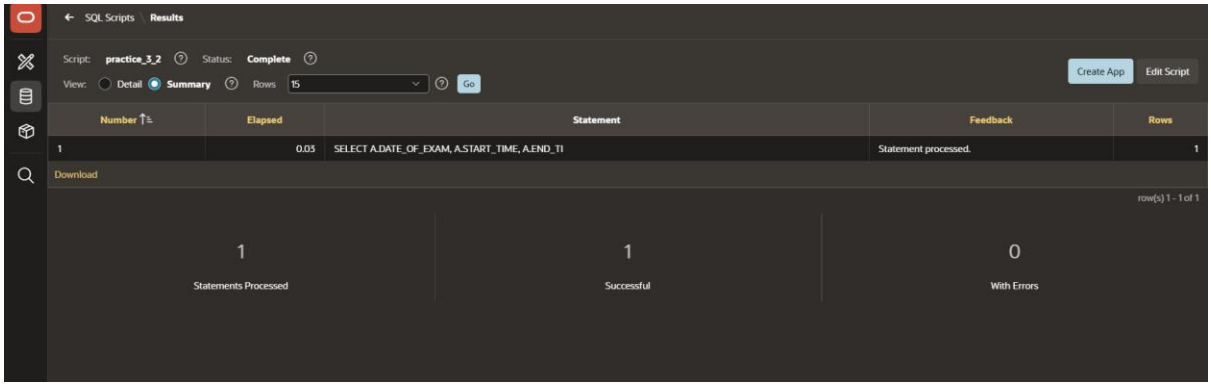
Tracking Data Changes digunakan untuk menyimpan perubahan informasi dari waktu ke waktu. Selain itu, time-related constraints digunakan untuk menjaga agar jadwal dan penggunaan ruangan tidak saling bertabrakan.

Implementasi Oracle Apex

Input salah satu gambar



Hasil (output):



PRACTICE 3-3

Exercise 1: Relational Databases

1. First Normal Form (1NF)

Tabel awal belum memenuhi 1NF karena atribut Color memiliki lebih dari satu nilai dalam satu kolom.

Solusinya adalah memisahkan warna sehingga setiap baris hanya memiliki satu nilai.

Tabel ITEM_COLOR

Item ID	Color
IT001	Red
IT001	Blue
IT002	Yellow
IT003	Green
IT004	Blue
IT004	Yellow

Tabel ITEM_PRICE

Item ID	Unit Price
IT001	$16.56
IT002	$17.48
IT003	$19.76
IT004	$20.00

Dengan demikian, setiap kolom memiliki nilai tunggal atau atomic value.

2. Second Normal Form (2NF)

Tabel awal memiliki Composite Primary Key, yaitu Supplier ID dan Store ID.

Masalahnya, Location hanya bergantung pada Store ID, bukan pada keseluruhan Primary Key.

Solusinya adalah memisahkan tabel menjadi:

SUPPLIER_STORE_DETAILS

- Supplier ID (PK, FK)
- Store ID (PK, FK)

STORE_LOCATION

- Store ID (PK)
- Location

Dengan pemisahan ini, Location bergantung sepenuhnya pada Primary Key Store ID dan tidak lagi memiliki partial dependency.

Catatan: Data awal mempunyai perbedaan Location untuk Store ID yang sama, yaitu S1 dan S3. Data tersebut perlu dikonfirmasi sebelum dimasukkan ke tabel STORE_LOCATION.

3. Third Normal Form (3NF)

Tabel awal memiliki transitive dependency karena Category Description bergantung pada Category ID, bukan secara langsung pada Book ID.

Solusinya adalah memisahkan menjadi:

BOOK_PRICE_DETAILS

- Book ID (PK)
- Category ID (FK)
- Price

CATEGORY_DETAILS

- Category ID (PK)
- Category Description

Dengan demikian, Category Description disimpan berdasarkan Category ID sehingga tidak terjadi transitive dependency.

pdfcoffee.com

Exercise 2: Normalize Academic Database ERD

Berdasarkan ERD Academic Database, terdapat tiga bagian yang perlu diperbaiki agar memenuhi aturan normalisasi.

1. STUDENT ATTENDANCE

Atribut Number of Working Days, Number of Days Off, dan Eligibility for Exam harus dipindahkan dari STUDENT ke entitas baru bernama STUDENT ATTENDANCE.

Alasannya, informasi kehadiran dapat berubah pada setiap Academic Session sehingga termasuk repeating group dan melanggar 1NF.

2. FACULTY LOGIN DETAIL

Atribut Login Date dan Login Time harus dipindahkan dari FACULTY ke entitas FACULTY LOGIN DETAIL.

Alasannya, seorang dosen dapat melakukan login berkali-kali sehingga informasi tersebut merupakan repeating group yang melanggar 1NF.

3. EXAM TYPE

Atribut Type, Name, dan Description perlu dipisahkan dari EXAM menjadi entitas EXAM TYPE.

Alasannya, Exam Type menentukan Name dan Description sehingga menimbulkan transitive dependency dan melanggar 3NF.

Kesimpulan Exercise 2:

Perbaikan Academic Database dilakukan dengan memindahkan atribut yang memiliki repeating group ke entitas baru dan memisahkan atribut yang memiliki transitive dependency.

Ketiga perubahan tersebut membuat struktur database lebih terorganisasi dan mengurangi pengulangan data.

pdfcoffee.com

Exercise 3: Validate an ERD for Normalization

1. USER Database

Tabel USER awal mengandung informasi pengguna, pesan, dan server dalam satu tabel. Hal ini menyebabkan pengulangan data seperti User Name dan Server Name.

Untuk mencapai 3NF, tabel dibagi menjadi empat entitas:

USER

- User ID (PK)
- User Name
- Server ID (FK)

MESSAGE

- Message ID (PK)
- Subject
- Text

SERVER

- Server ID (PK)
- Server Name

RECEIVED MESSAGE

- User ID (PK, FK)
- Message ID (PK, FK)
- Recorded Date

Penjelasan:

- 1NF: Memisahkan informasi USER dan MESSAGE agar setiap pengguna dapat menerima banyak pesan.
- 2NF: Membuat RECEIVED MESSAGE sebagai entitas penghubung untuk menyimpan Recorded Date.
- 3NF: Memisahkan SERVER karena Server Name bergantung pada Server ID.

2. CAR Entity

Model CAR awal belum memenuhi 3NF karena Paint Color dan Interior Color sebenarnya bergantung pada Color Scheme.

Solusinya adalah membuat entitas baru bernama COLOR SCHEME.

CAR

- Car Number (PK)
- Make
- Model
- Scheme Code (FK)

COLOR SCHEME

- Scheme Code (PK)
- Scheme Name

- Paint Color
- Interior Color

Dengan demikian, setiap informasi warna disimpan pada entitas COLOR SCHEME dan dapat digunakan oleh beberapa kendaraan.

pdfcoffee.com

Exercise 4: Gather Database Requirements and Business Rules

1. Business Rules – Book.com

Berdasarkan studi kasus Book.com, aturan bisnis yang dapat diidentifikasi adalah:

1. CUSTOMER dapat memiliki nol, satu, atau banyak ORDER.
2. CUSTOMER_ADDRESS hanya dapat terhubung dengan satu ADDRESS.
3. CUSTOMER_ADDRESS hanya dapat dimiliki oleh satu CUSTOMER.
4. CUSTOMER wajib memiliki EMAIL ADDRESS.
5. DELIVERY harus terhubung dengan satu ORDER.
6. ORDER harus dimiliki oleh satu CUSTOMER.
7. ORDER harus memiliki satu atau lebih PRODUCT.
8. PAYMENT harus terhubung dengan satu ORDER.
9. ORDER harus berisi PRODUCT yang valid.

2. Classification of Business Rules

Berikut klasifikasi setiap aturan bisnis berdasarkan jenisnya.

Business Rule	Jenis
All teachers must possess a valid teaching certificate.	Structural
Each Department must offer a Course.	Structural
Travel requests must be approved by the project manager.	Procedural
A customer may make numerous payments on account.	Structural
A machine operator may not work more than 10 hours a day.	Programmatic
Rental amount = rental rate × number of days.	Programmatic
A Customer can have zero, one or many ORDERS.	Structural
Total rental cost = insurance + rental amount + late charge.	Programmatic
Customer's debt must not exceed the credit limit.	Programmatic

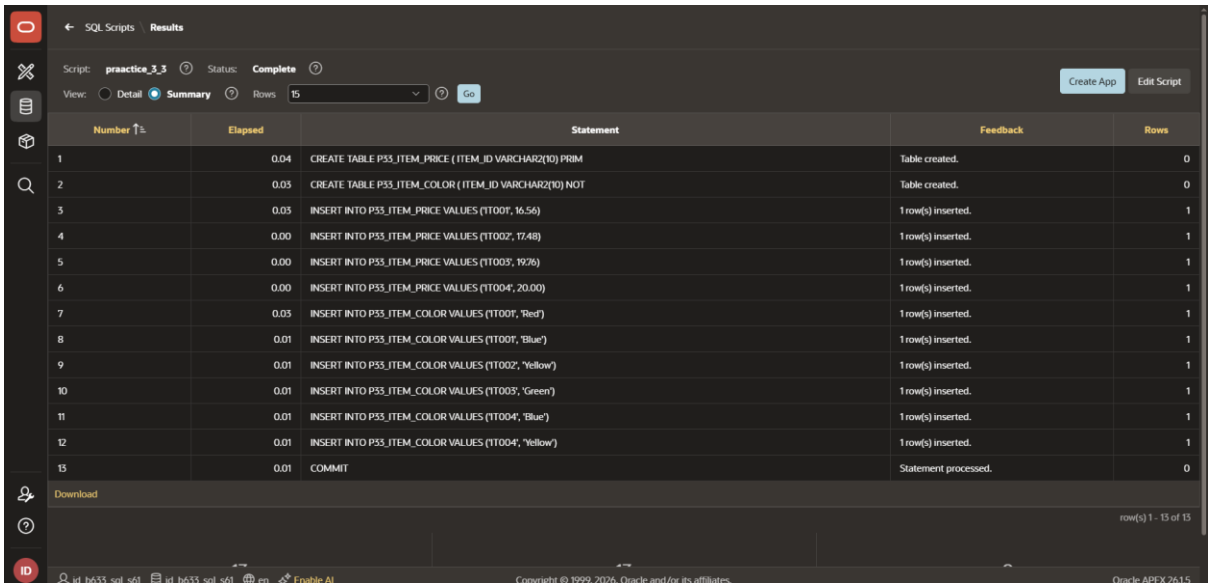
Kesimpulan:

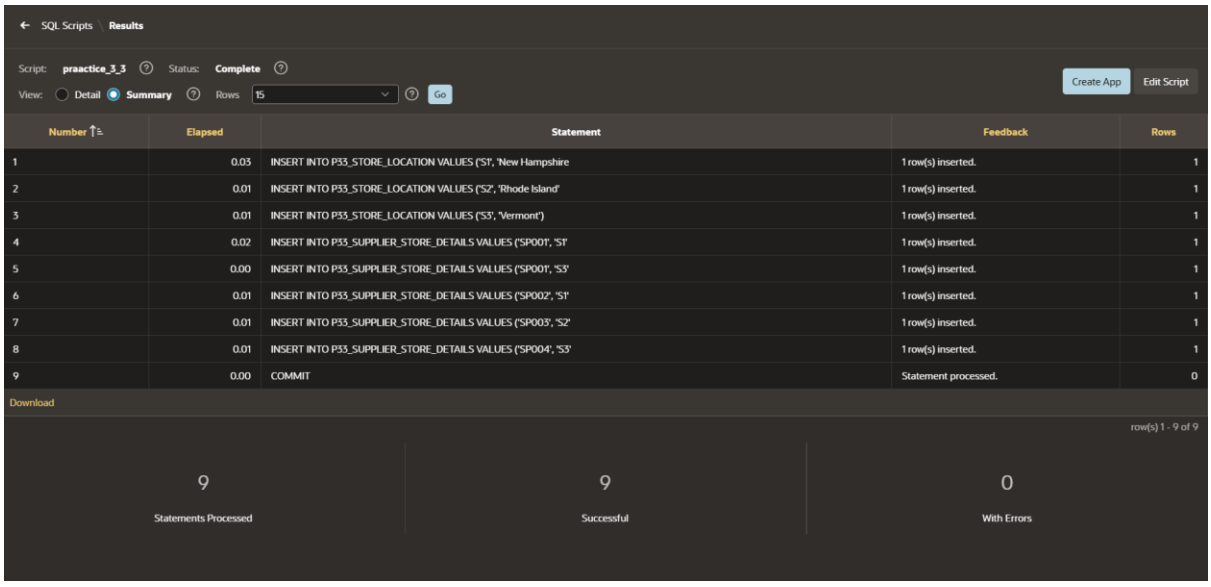
Structural Business Rules menjelaskan struktur data dan hubungan antarentitas. Procedural Business Rules mengatur proses atau prosedur kerja. Programmatic Business Rules merupakan aturan yang membutuhkan perhitungan atau validasi melalui program.

Implementasi Oracle Apex

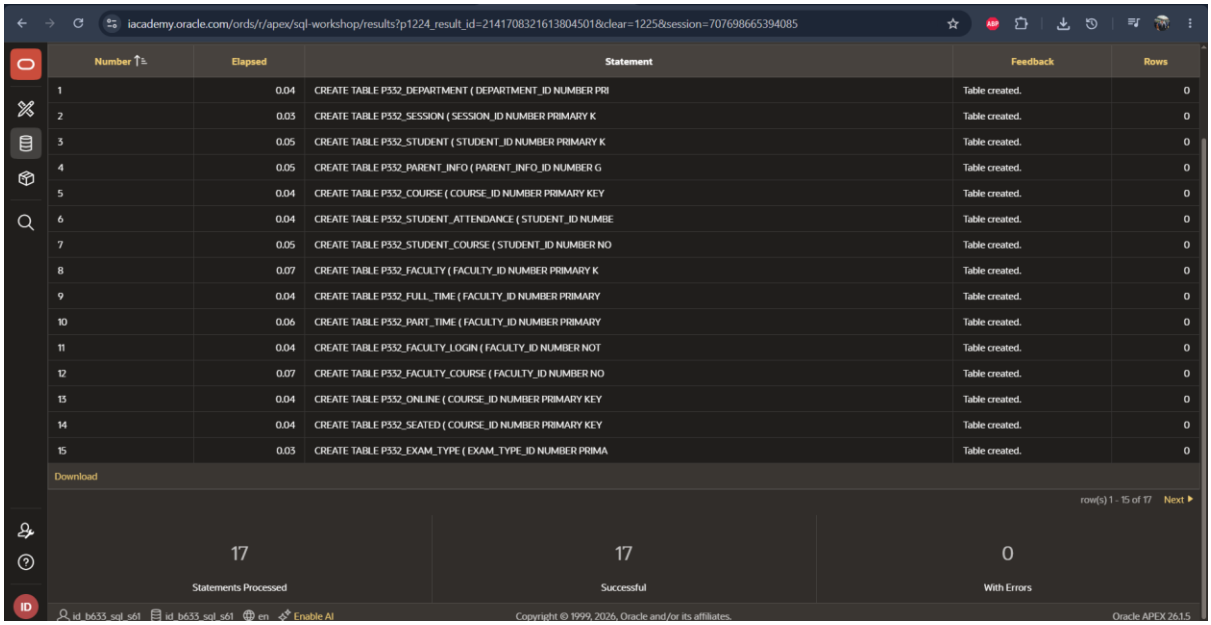
Input salah satunya:

Exercise 1

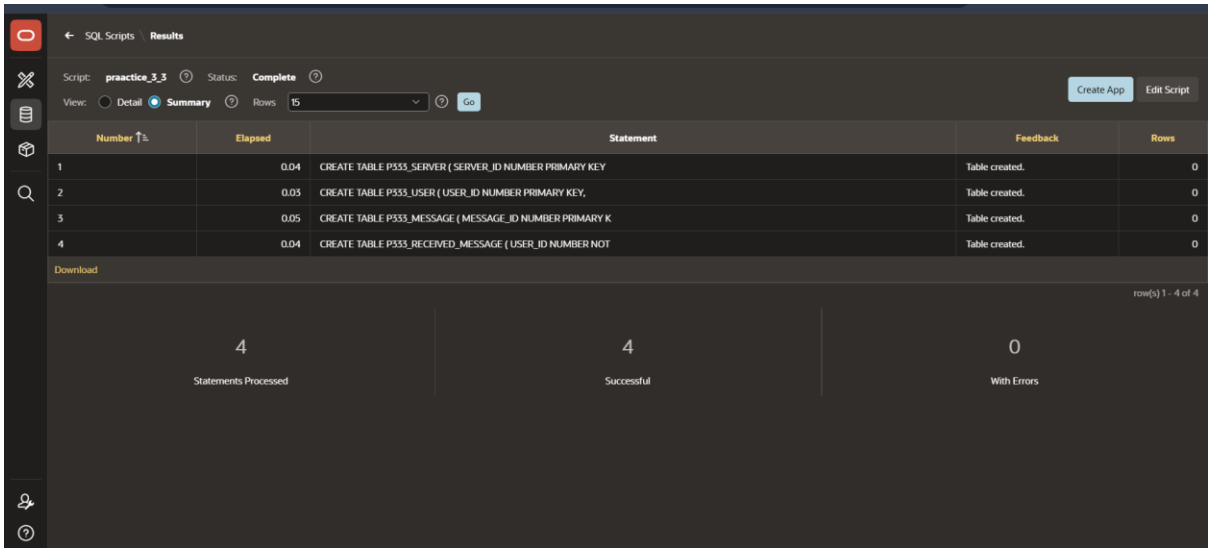




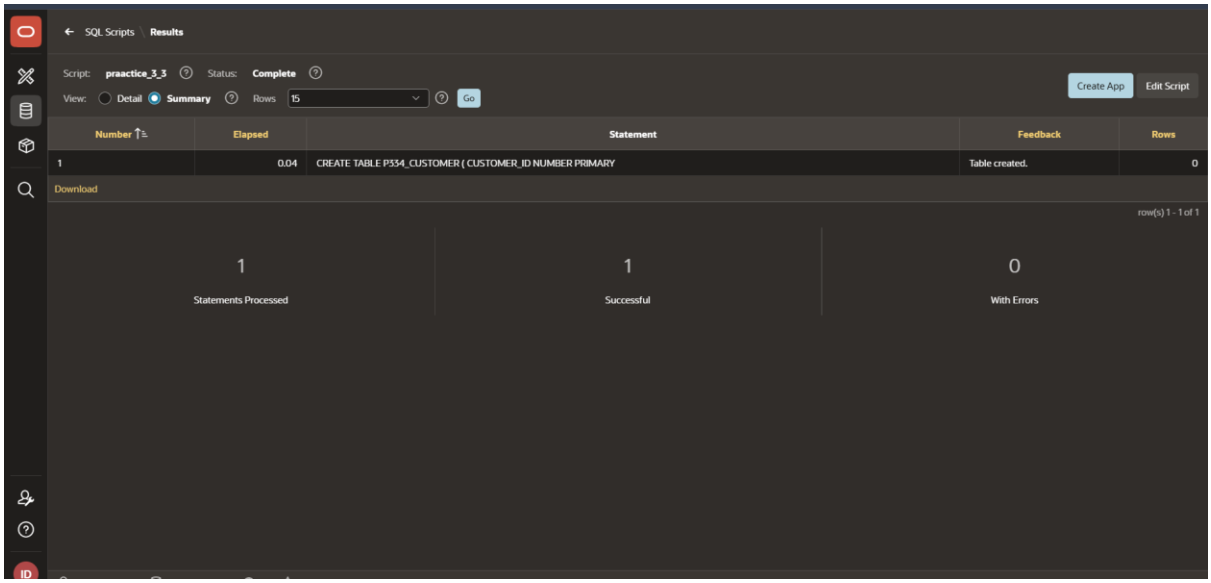
Exercise 2



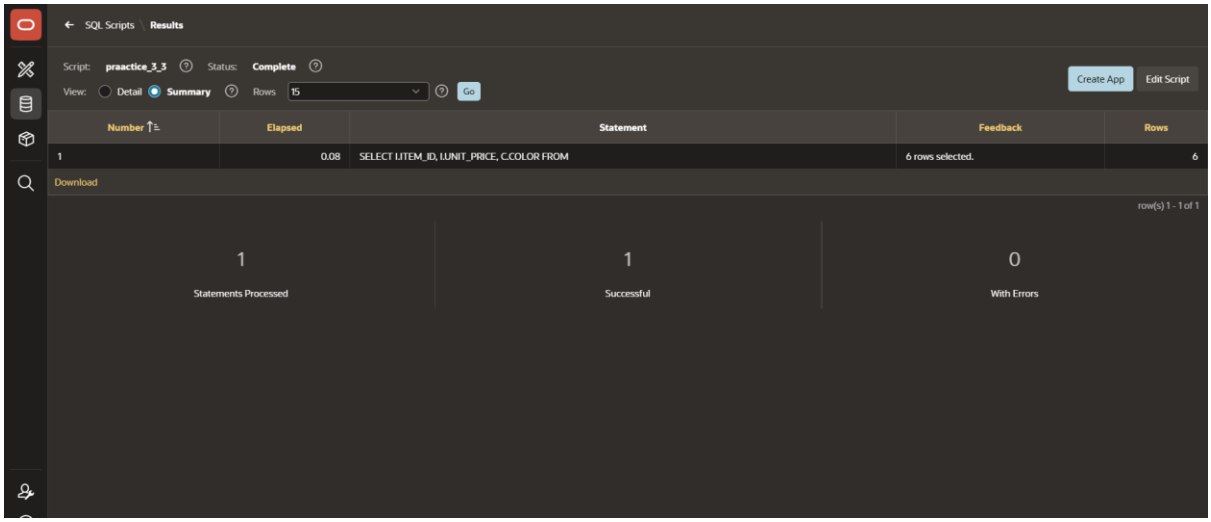
EXERCISE 3



EXERCISE 4



HASIL OUTPUT:



PRACTICE 3-4

Exercise 1: Identify Entities, Attributes, Instances and Their Corresponding Tables, Rows and Columns

1. Match the ERD Elements to Their Corresponding Database Elements

No.	Analysis	Design
1	Attribute	a. Column
2	Entity	f. Table

3	ER Model	c. Physical Design
4	Instance	e. Row
5	Primary UID	d. Primary Key
6	Relationship	b. Foreign Key
7	Secondary UID	g. Unique Key

2. Identify the Table Diagram Notations

- `PK` (Primary Key): Atribut yang menjadi identitas unik setiap record dalam tabel.
- `FK` (Foreign Key): Atribut yang menghubungkan satu tabel dengan tabel lainnya.
- `UK` (Unique Key): Atribut yang nilainya harus unik dan tidak boleh sama dengan record lainnya.
- `*` (Mandatory): Atribut wajib diisi atau tidak boleh NULL.
- `°` (Optional): Atribut boleh dikosongkan atau NULL.

3. Create Short Names

Berikut nama singkat berdasarkan konvensi penamaan database:

a. Authors → `AUTH`

b. Publishers → `PUBL`

c. Customers → `CUST`

Nama singkat dibuat menggunakan huruf kapital, mudah dikenali, dan konsisten agar mempermudah penamaan tabel maupun atribut database.

4. Identify Attributes for SONG, EVENT, and CUSTOMER

Attributes	SONG	EVENT	CUSTOMER
Title	X	X	
Description	X	X	
Venue		X	
First Name			X
Phone Number			X
Release Date	X		
Last Name			X
Type	X	X	
Email Address			X

Keterangan: X menunjukkan bahwa atribut dapat digunakan pada entitas tersebut. Type pada SONG dapat berupa genre, sedangkan Type pada EVENT dapat berupa jenis acara.

Exercise 2: Mapping the Academic Database

1. Mapping Entities and Attributes

Mapping merupakan proses mengubah rancangan ERD menjadi struktur database yang terdiri dari tabel dan kolom.

Berikut hasil mapping Academic Database:

Entity	Table Name	Primary Key
PARENT INFORMATION	PARENT_INFO	parent_info_id
STUDENT	STUDENT	student_id
STUDENT ATTENDANCE	STUDENT_ATTENDANCE	student_id + session_id
STUDENT COURSE DETAIL	STUDENT_COURSE_DETAIL	student_id + course_id + session_id
DEPARTMENT	DEPARTMENT	department_id
COURSE	COURSE	course_id
ONLINE	ONLINE	course_id
SEATED	SEATED	course_id
ACADEMIC SESSION	ACADEMIC_SESSION	session_id
EXAM RESULT	EXAM_RESULT	student_id + exam_id
FACULTY	FACULTY	faculty_id
FULL TIME	FULL_TIME	faculty_id
PART TIME	PART_TIME	faculty_id
FACULTY COURSE DETAIL	FACULTY_COURSE_DETAIL	faculty_id + course_id + session_id
FACULTY LOGIN DETAIL	FACULTY_LOGIN_DETAIL	faculty_id + login_date_time
EXAM	EXAM	exam_id
EXAM TYPE	EXAM_TYPE	exam_type_id

Selain tabel tersebut, ditambahkan tabel penghubung COURSE_SESSION untuk mencatat hubungan antara COURSE dan ACADEMIC SESSION.

2. Mapping Attributes

Setiap atribut pada ERD akan menjadi kolom pada tabel database.

Contohnya:

- Entity STUDENT menjadi tabel STUDENT.
- Attribute First Name menjadi kolom FIRST_NAME.
- Attribute Last Name menjadi kolom LAST_NAME.
- Attribute Email menjadi kolom EMAIL.
- Primary UID menjadi Primary Key.
- Relationship antarentitas diterapkan menggunakan Foreign Key.

3. Mapping Instances

Instance merupakan data nyata yang disimpan dalam tabel.

Contohnya, pada tabel STUDENT terdapat data mahasiswa bernama Andi dengan Student ID 1001. Data tersebut disebut satu instance atau satu row.

Sedangkan FIRST_NAME, LAST_NAME, dan EMAIL merupakan columns dari tabel STUDENT.

Kesimpulan

Berdasarkan Practice 3-4, dapat dipahami bahwa ERD merupakan rancangan database yang dapat diterapkan menjadi tabel fisik.

Entity dipetakan menjadi table, attribute menjadi column, instance menjadi row, Primary UID menjadi Primary Key, dan relationship diterapkan menggunakan Foreign Key.

Dengan proses mapping, rancangan database dapat diimplementasikan ke dalam Oracle Database secara lebih terstruktur.

Implementasi Oracle Apex

Input:

SQL Scripts \ Script Editor

Script Name: practice_3_4

Cancel | Save | Run

```
-- 1. DEPARTMENT
CREATE TABLE P34_DEPARTMENT (
    DEPARTMENT_ID NUMBER PRIMARY KEY,
    NAME VARCHAR2(150) NOT NULL,
    HEAD VARCHAR2(150) NOT NULL
);

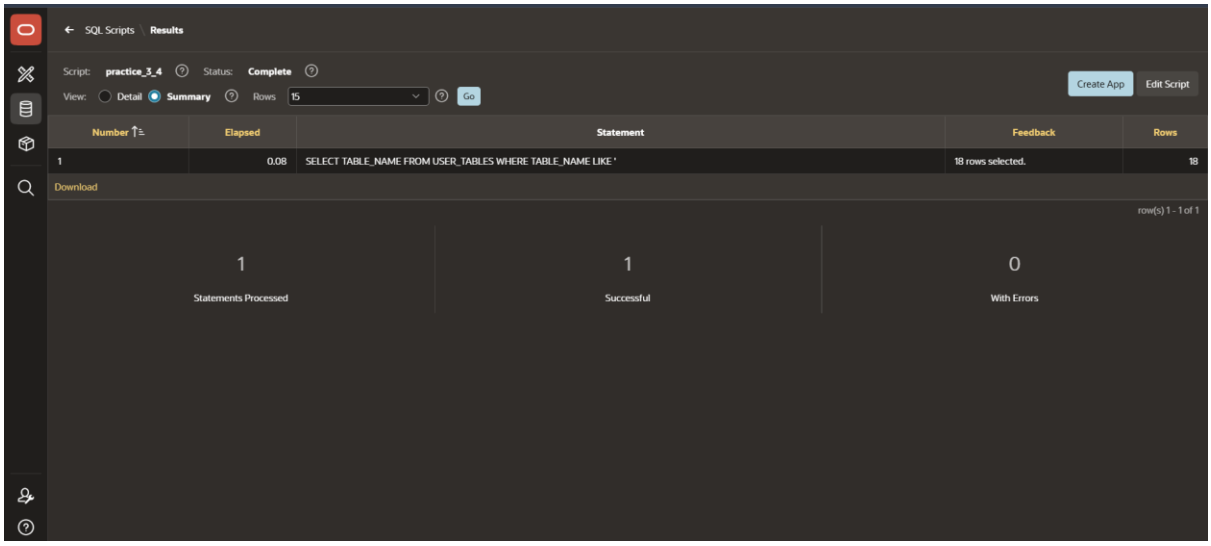
-- 2. ACADEMIC SESSION
CREATE TABLE P34_ACADEMIC_SESSION (
    SESSION_ID NUMBER PRIMARY KEY,
    NAME VARCHAR2(100) NOT NULL
);

-- 3. STUDENT
CREATE TABLE P34_STUDENT (
    STUDENT_ID NUMBER PRIMARY KEY,
    FIRST_NAME VARCHAR2(100) NOT NULL,
    LAST_NAME VARCHAR2(100) NOT NULL,
    REGISTRATION_YEAR NUMBER(4) NOT NULL,
    EMAIL VARCHAR2(150) UNIQUE NOT NULL
);

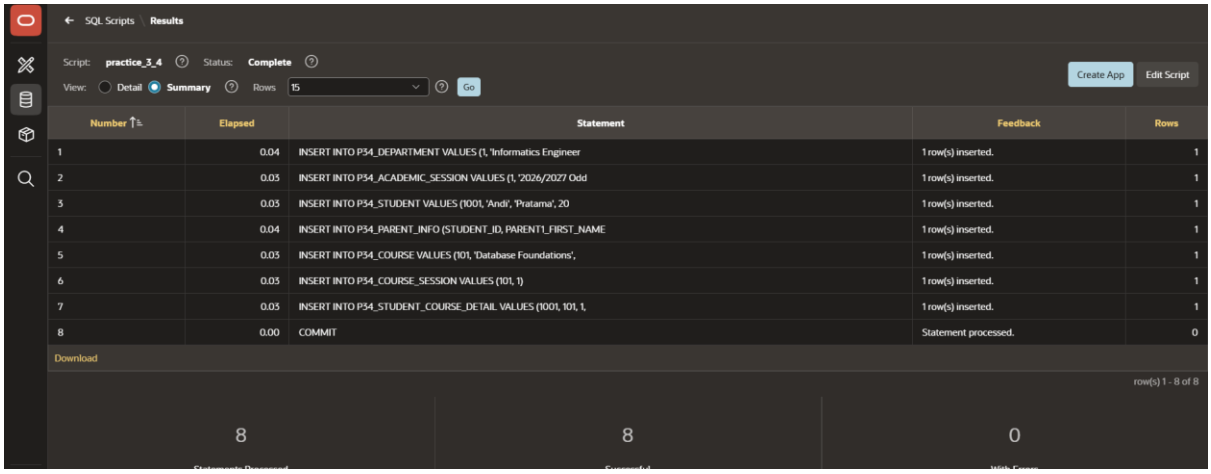
-- 4. PARENT INFORMATION
CREATE TABLE P34_PARENT_INFO (
    PARENT_INFO_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    STUDENT_ID NUMBER NOT NULL UNIQUE
        REFERENCES P34_STUDENT(STUDENT_ID),
    PARENT1_FIRST_NAME VARCHAR2(100) NOT NULL,
    PARENT1_LAST_NAME VARCHAR2(100) NOT NULL,
    PARENT2_FIRST_NAME VARCHAR2(100),
    PARENT2_LAST_NAME VARCHAR2(100)
);

-- 5. STUDENT ATTENDANCE
CREATE TABLE P34_STUDENT_ATTENDANCE (
```

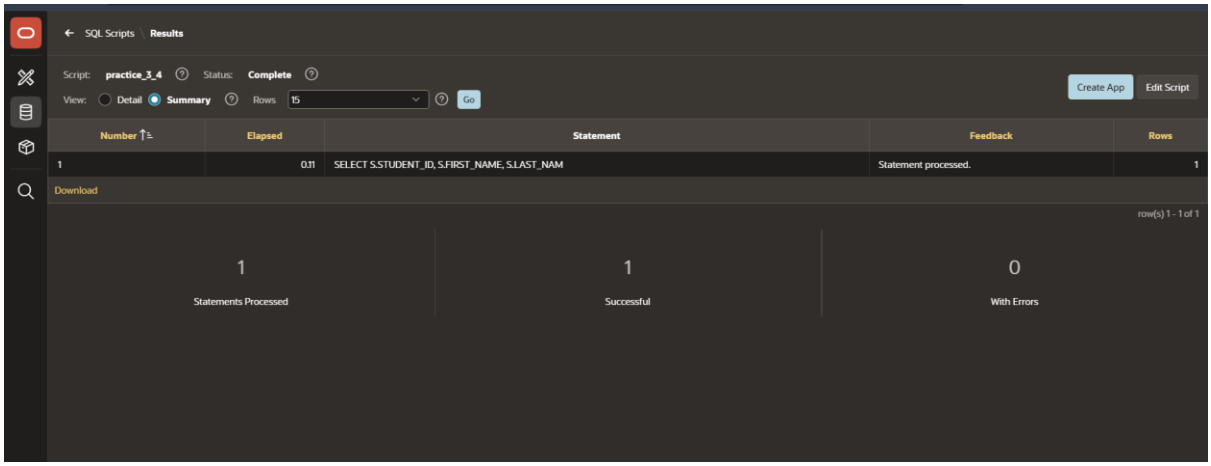
Output:



Input:



Output:



5. SECTION-4

PRACTICE 4-1

Exercise 1: Installing Oracle SQL Developer Data Modeler

Tujuan: Menginstal aplikasi Oracle SQL Developer Data Modeler untuk membuat rancangan database secara visual.

Untuk Windows 64-bit, langkah instalasinya:

1. Buka halaman resmi Oracle SQL Developer Data Modeler Downloads.
2. Pilih paket Windows 64-bit.
3. Download file ZIP.
4. Extract file ke folder yang diinginkan.
5. Buka folder hasil ekstraksi.
6. Jalankan `datamodeler64.exe`.
7. Tunggu hingga aplikasi terbuka.

Informasi versi: halaman resmi Oracle yang diperiksa menampilkan Data Modeler 24.3.1. Paket Windows 64-bit tersebut sudah menyertakan JDK 17, sehingga tidak perlu memasang JDK secara terpisah.

Oracle

Exercise 2: Identify and Create Entities, Attributes and Relationships

Tujuan: Membuat rancangan Academic Database atau School Management System dengan menentukan entitas, atribut, dan hubungan antarentitas menggunakan Oracle SQL Developer Data Modeler.

Entitas yang digunakan:

Entity	Attributes
PARENT_INFORMATION	# parent_info_id, * student_id, * parent1_first_name, * parent1_last_name, ° parent2_first_name, ° parent2_last_name
STUDENT	# student_id, * department_id, * first_name, * last_name, * registration_year, * email
STUDENT_ATTENDANCE	# student_id, # session_id, * number_of_working_days, * number_of_days_off, ° eligibility_for_exam
STUDENT_COURSE_DETAIL	# student_id, # course_id, # session_id, * grade
DEPARTMENT	# department_id, * name, * head
COURSE	# course_id, * department_id, * name
ONLINE	# course_id, * logon_id, * password
SEATED	# course_id, * building, * room, * date_time
ACADEMIC_SESSION	# session_id, * name
EXAM_RESULT	# student_id, # exam_id, * grade
FACULTY	# faculty_id, * department_id, * first_name, * last_name, * email
FULL_TIME	# faculty_id, * salary, * insurance
PART_TIME	# faculty_id, * hourly_rate
FACULTY_COURSE_DETAIL	# faculty_id, # course_id, # session_id, * contact_hours
FACULTY_LOGIN_DETAIL	# faculty_id, # login_date_time, ° logout_date_time
EXAM	# exam_id, * course_id, * session_id, * exam_type_id, ° start_date
EXAM_TYPE	# exam_type_id, * type, * name, ° description

Keterangan: # menunjukkan Unique Identifier, * Mandatory, dan ° Optional. Foreign Key digunakan untuk menghubungkan entitas.

Hubungan antarentitas:

- PARENT_INFORMATION terhubung dengan STUDENT.
- STUDENT terhubung dengan STUDENT_ATTENDANCE.
- STUDENT terhubung dengan STUDENT_COURSE_DETAIL.
- STUDENT terhubung dengan EXAM_RESULT.
- DEPARTMENT terhubung dengan COURSE dan FACULTY.
- COURSE terhubung dengan STUDENT_COURSE_DETAIL.
- COURSE terhubung dengan FACULTY_COURSE_DETAIL.
- COURSE memiliki subtype ONLINE dan SEATED.
- FACULTY terhubung dengan FACULTY_COURSE_DETAIL dan FACULTY_LOGIN_DETAIL.
- FACULTY memiliki subtype FULL_TIME dan PART_TIME.
- ACADEMIC_SESSION terhubung dengan STUDENT_ATTENDANCE, COURSE, dan EXAM.

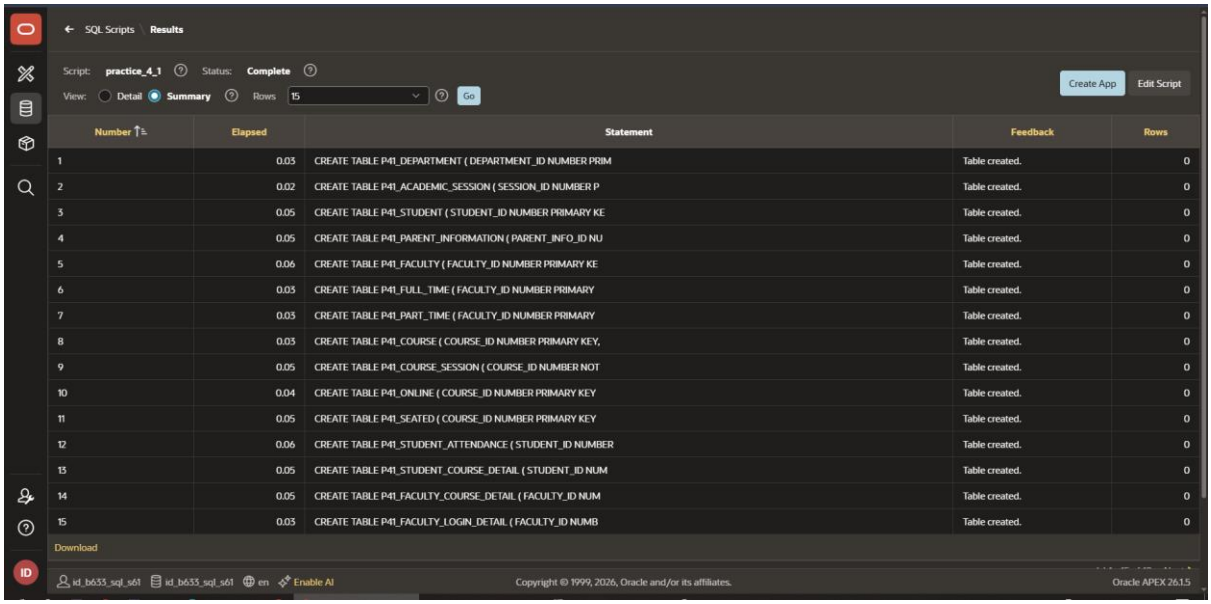
- EXAM_RESULT terhubung dengan EXAM.
- EXAM terhubung dengan EXAM_TYPE.

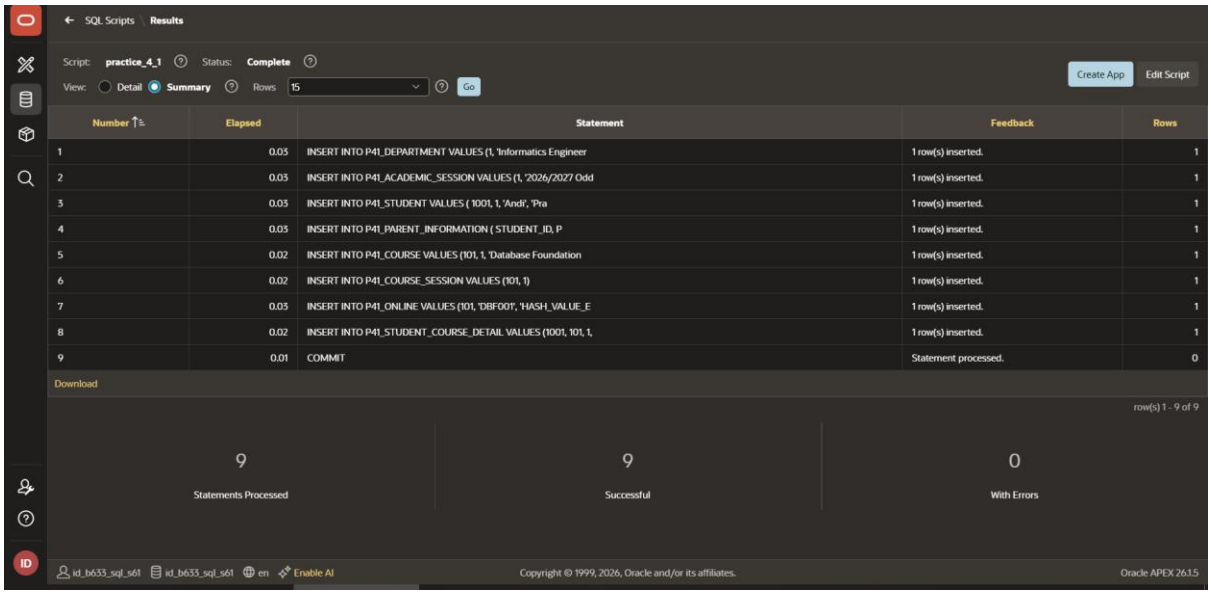
Kesimpulan:

Oracle SQL Developer Data Modeler dapat digunakan untuk membuat rancangan database melalui entitas, atribut, Unique Identifier, dan hubungan antarentitas. Rancangan tersebut dapat dikembangkan menjadi Relational Model dan menghasilkan SQL untuk diterapkan ke Oracle Database.

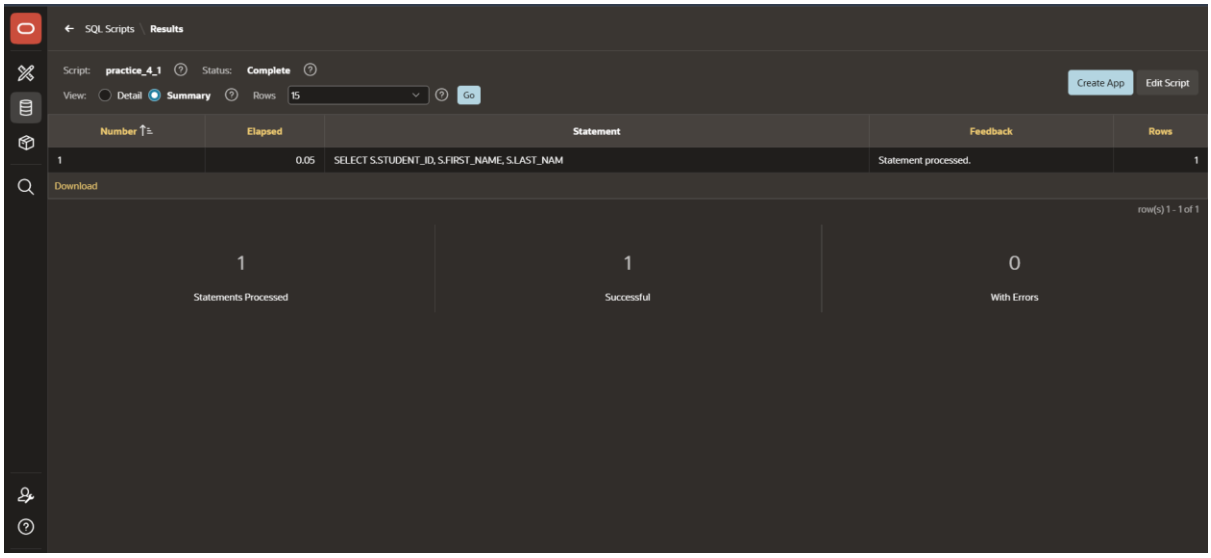
Implementasi Oracle Apex

Input salah satu contoh:





HASIL Output:



PRACTICE 4-2

Exercise 1: Forward Engineer a Logical Model to a Relational Model

1. Tujuan Praktikum

Praktikum ini bertujuan untuk mengubah Logical Model Academic Database menjadi Relational Model menggunakan Oracle SQL Developer Data Modeler.

Logical Model menggambarkan entitas, atribut, dan hubungan antarentitas secara konseptual. Sedangkan Relational Model menggambarkan struktur database dalam bentuk tabel, kolom, Primary Key, dan Foreign Key.

2. Langkah Pengerjaan

Proses forward engineering dilakukan dengan tahapan berikut:

1. Membuka file Logical Model Academic Database yang telah dibuat pada Practice 4-1.
2. Membuka tampilan Logical Model.
3. Memilih ikon Engineer to Relational Model pada toolbar.
4. Setelah jendela Engineering muncul, membiarkan pengaturan menggunakan konfigurasi default.
5. Klik tombol Engineer.
6. Membuka bagian Relational Models pada Object Browser.

7. Memeriksa objek relational yang berhasil dibuat.
8. Menyimpan hasil rancangan database.

3. Hasil Forward Engineering

Setelah proses berhasil, Logical Model akan dikonversi menjadi Relational Model.

Hasil konversinya meliputi:

- Entity menjadi Table.
- Attribute menjadi Column.
- Unique Identifier menjadi Primary Key.
- Relationship menjadi Foreign Key.
- Many-to-Many Relationship dikonversi menggunakan tabel penghubung.

Relational Model tersebut menjadi rancangan struktur database yang dapat digunakan untuk menghasilkan SQL (DDL).

Oracle Documentation+1

4. Kesimpulan

Berdasarkan praktikum 4-2, dapat dipahami bahwa proses Forward Engineering digunakan untuk mengubah Logical Model menjadi Relational Model.

Dengan menggunakan Oracle SQL Developer Data Modeler, rancangan database yang sebelumnya berbentuk konseptual dapat dikonversi menjadi struktur tabel yang lebih detail dan siap dikembangkan ke tahap implementasi database.

Link gith : https://github.com/yafi206/Tugas-Oracle-Academy-Chapt-1-4

Link drive : https://drive.google.com/drive/folders/1OBvurnYorpL9b7wwbqet-cDzq3TmHrMd?usp=drive_link